Holidays in New York





Images were inpainted and manipulated with $1600

What is the best small AI model for inpainting-based synthetic image detection?

Final presentation

12/10/2026

Pr Adrian Popescu

Wissam AMAR • Hasti HOSSEINPOUR • Dominika CHOJNACKA • Youssef EL OTMANI • Sathya GNANOU

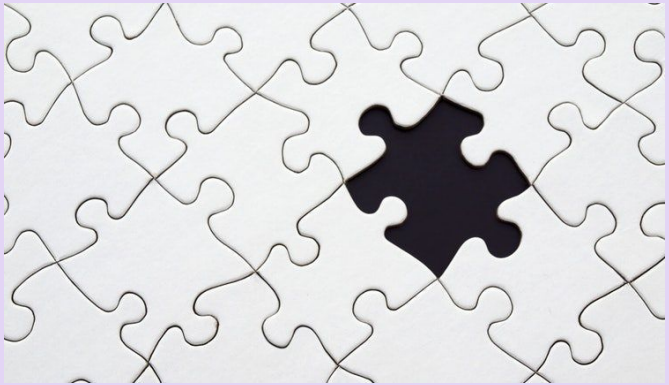
Inpainting

restoring missing or damaged regions of an image using information from the surrounding pixels



The Gap

Heterogeneous experimental settings, difficult to compare selected small models across performance, generalization, and computational efficiency.



The need for a **unified benchmarking framework**

Research Question

RQ1 What are the categories or models and different architectures used to detect / localize inpainting?

RQ2 Which evaluation criteria are appropriate for comparing small AI models for inpainting detection and localization?

RQ3 How do selected models compare under a standardized evaluation protocol?

RQ4 What trade-offs exist between detection / localization performance and computational cost among selected models?

Criteria to choose the models

1. **Different families:** one model per approach (CNN, state space, transformer, reconstruction).
2. **Efficiency:** lightweight or low-compute design, since our question is about small models.
3. **Both tasks, or adaptable:** detection and localization.
4. **Relevant:** built for image manipulation, ideally for generated edits.

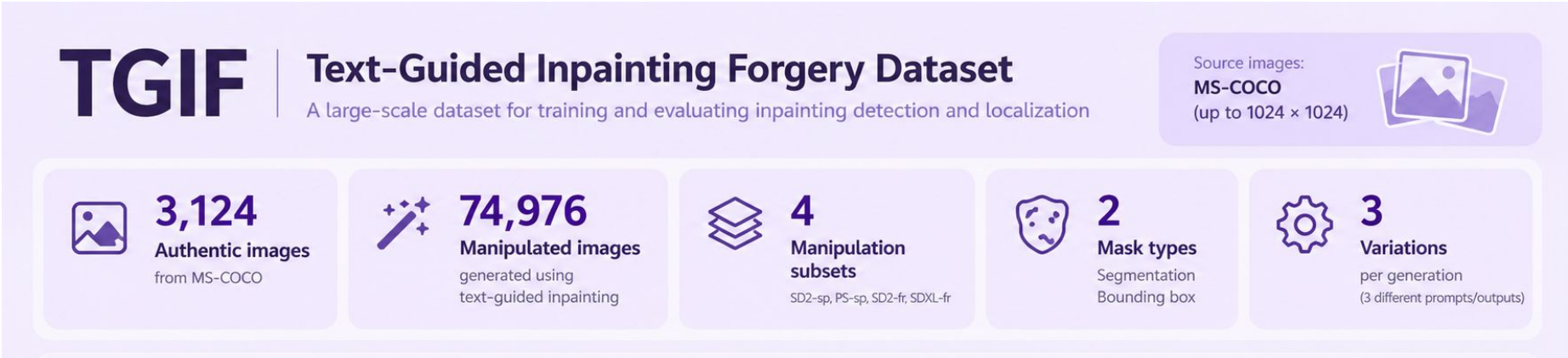
Detection and localization architectures

CNN-based	PSCC-Net
Transformer-based	TruFora
Reconstruction-based	AEROBLADE
State Space Models	ForMa

Dataset: TGIF

Why is it relevant ?

- Large-scale dataset
- Diversity of generative methods
- Relevance to our research question
- Recent and established benchmark



Dataset: TGIF

Why is it relevant ?

It is compliant with the **F.A.I.R** principles

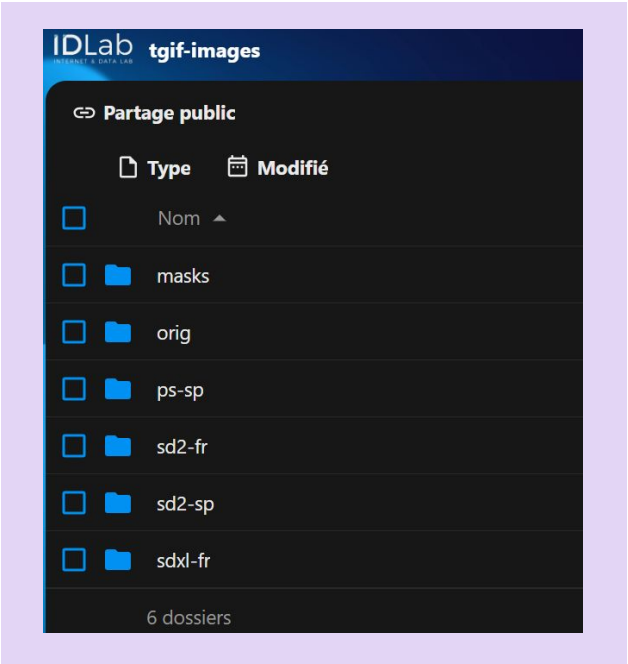
F Findable: Data and metadata must be easy to locate

A Accessible: Once data is found, users must know how to retrieve it.

I Interoperable: Data must integrate and work well with other datasets, storage systems, or analytical tools.

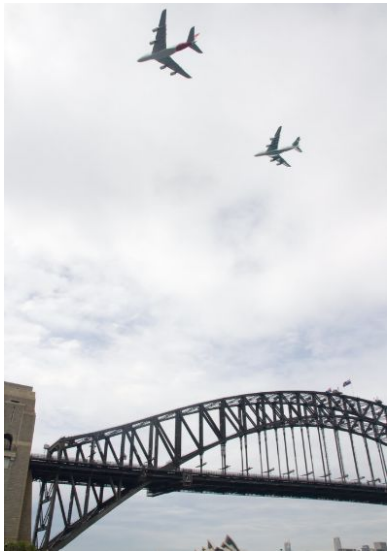
R Reusable: Data must be documented well enough to be replicated and used in new studies

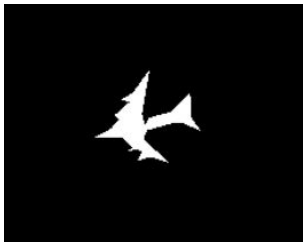
More about the TGIF Dataset

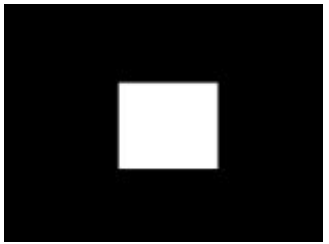


We focus on splicing and not fully regenerated images: either with photoshop and adobe firefly or stable diffusion

Inpainted Input Image







Masks: Localization Ground Truth

Evaluation Metrics

Detection

- Accuracy
- F1 score
- ROC-AUC

Localization

- IoU
- Dice Score
- Pixel-wise Recall
- Pixel-wise Precision

Efficiency Metrics : Compute power, Model Size, trade-off

Evaluation Metrics

Visualizing Localisation Metrics



Ground Truth Mask (G)

$|G| = 100$ pixels



Predicted Mask (P)

$|P| = 120$ pixels



Intersection ($P \cap G$)

$|P \cap G| = 80$ pixels



Union ($P \cup G$)

$|P \cup G| = 140$ pixels

IoU

$$\frac{80}{140} = 0.571$$

(57.1%)

Dice Score

$$\frac{2 \times 80}{100 + 120} = 0.727$$

(72.7 %)

Results

General Results: DETECTION

Combining all 4,116 images across 13 categories and both generators (PS and SD)

	TrueFor	ForMa	PSCC-Net	AEROBLADE
Accuracy	0.95	0.77	0.66	0.57
precision	0.98	0.93	0.86	0.92
recall	0.96	0.82	0.69	0.59
F1	0.97	0.87	0.76	0.70
ROC-AUC	0.97	0.48	0.61	0.51

General Results: Localization

Combining all 4,116 images across 13 categories and both generators (PS and SD)

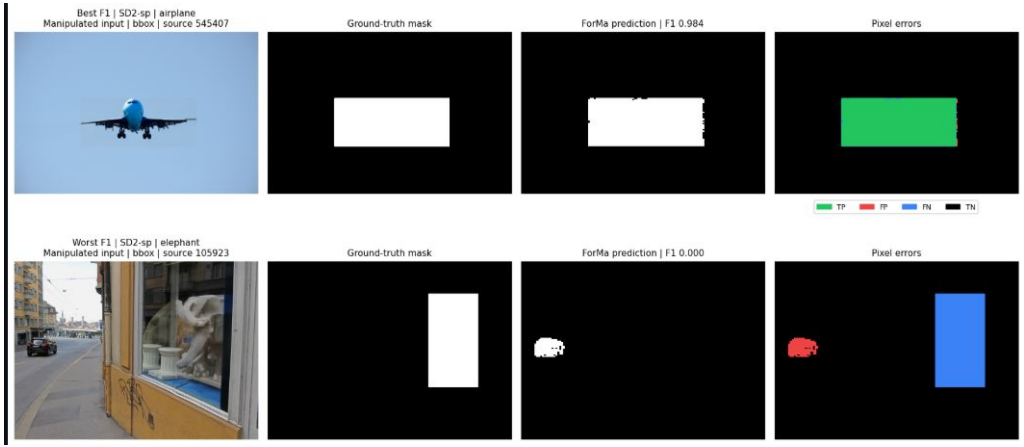
	TrueFor	PSCC-Net	ForMa	AEROBLADE
IoU	0.72	0.33	0.21	0.07
Dice	0.87	0.41	0.29	0.13
precision	0.93	0.46	0.41	0.08
recall	0.82	0.48	0.30	0.70

General Results: Time

Model	Params	GFLOPs / image	Latency / image (GPU, median)	Throughput	Peak RAM	Peak GPU mem	Est. full TGIF test (4,459 img)
AEROBLADE	98.4 M	3,841	891 ms (mean 833)	1.20 img/s	1,160 MiB	1,351 MiB	**≈ 62 min**
TruFor	68.7 M	1,255	872 ms (mean 836)	1.20 img/s	992 MiB	2,395 MiB	**≈ 62 min**
ForMa	37.3 M		11,628 ms (mean 11,659)	0.09 img/s	1679 MiB	446 MiB	**≈ 14.4 h**
PSCC-Net	3.7 M	542	856 ms (mean 897)	1.12 img/s	2,796 MiB	2,527 MiB	**≈ 67 min**

General Observed Patterns

- Localization is more sensitive to changes in inpatining method (PS, SD2)
- Localization and detection is generally more difficult on small objects than big items

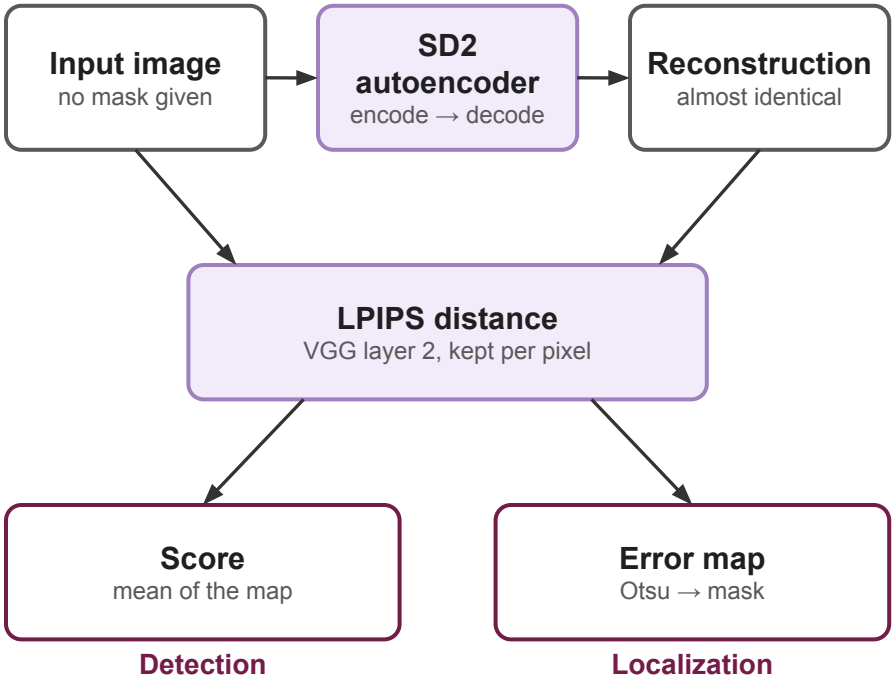


Model Specific Discussion



AEROBLADE: Reconstruction-Based Detection and Localization

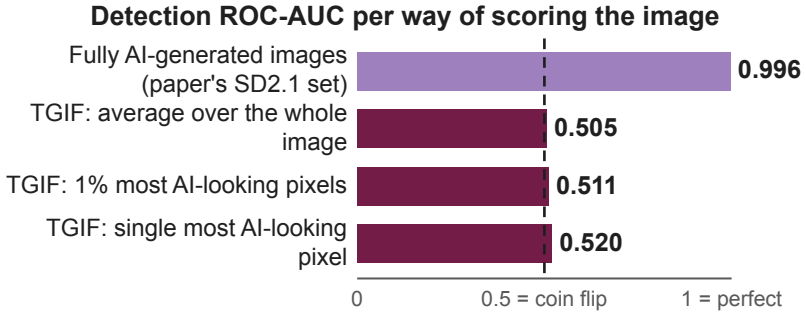
- Training-free: 1 autoencoder pass
- Low error = generated
- Mean of the map → detection
- Map itself → localization (our addition)
- SD2 autoencoder, ≈84M parameters



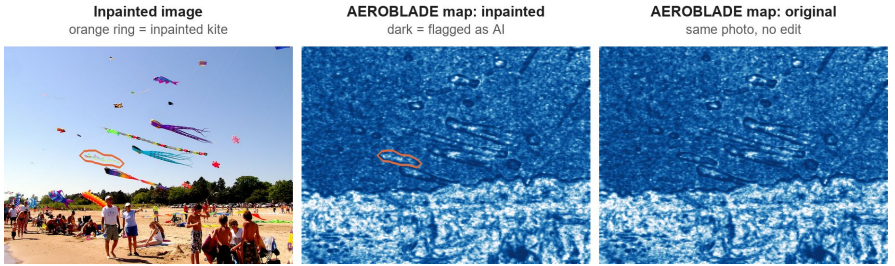
All 4,459 TGIF test images · 512×512 · laptop CPU, 18.4 s/image

Results: detection at chance, localization weak

Detection ROC-AUC 0.51 (0.5 = coin flip)



Localization IoU 0.07 · Dice 0.13 (1 = perfect)



AI-likeness: **real sky 0.82** · **inpainted kite 0.60**

Why?

- Every way of scoring the image is at chance
- Even the single most AI-looking pixel: 0.52
- Inpainted pixels don't look more AI than real ones (see the kite)

Why?

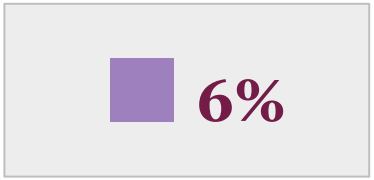
- No averaging here: one value per pixel
- The real sky is flagged, even in the untouched original
- Smooth areas rebuild easily → look AI: the map shows content, not the edit

Fully AI-generated: paper's own SD2.1 images vs its 1,000 real photos (our code). TGIF: all 4,459 test images. AI-likeness: map value (0–1) averaged over the sky (top third) and over the kite.

Why AEROBLADE fails on TGIF: 3 reasons

Our code is correct: same AP as the paper on the paper's own data (≥ 0.99).

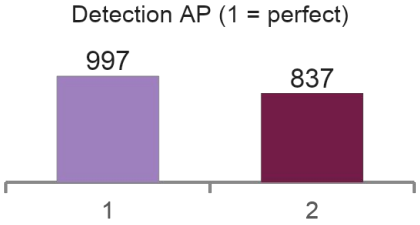
1 The edit is tiny



grey = real photo · purple = inpainted part

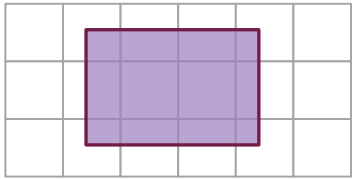
Only 6% of the image is AI-made, and it doesn't look more "AI" than the rest.

2 Resizing blurs the trace



Paper's own Midjourney images: AP 0.997 → 0.837 after our 512×512 resize.

3 The patch is off the 8×8 grid



squares = autoencoder blocks · purple = pasted edit

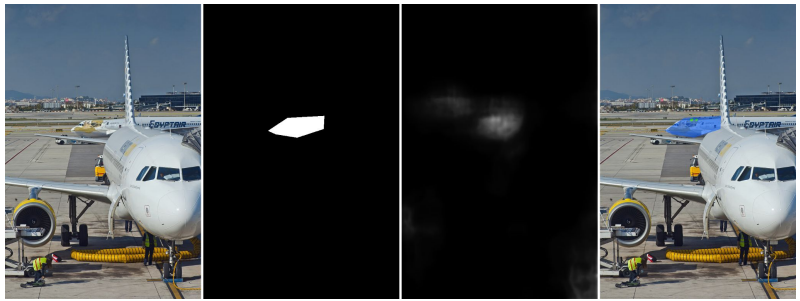
TGIF pastes the edit between the autoencoder's blocks.
Re-aligned*: ROC-AUC 0.66 → 0.82.

* uses TGIF's recorded paste position (upper bound) · 40 images at native size · pixel ROC-AUC 0.5 = chance

PSCC-Net - localization threshold

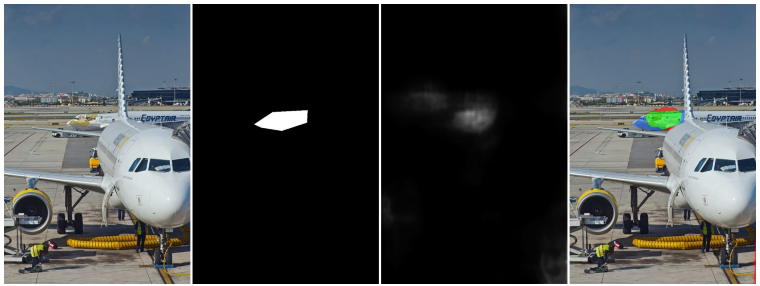
0.5

IoU	0.21
Dice	0.27
Precision	**0.45**
Recall	**0.24**



0.2

IoU	0.33
Dice	0.41
Precision	**0.46**
Recall	**0.48**



Conclusion: A lower threshold doubles recall without losing precision: PSCC-Net detects forgeries but is under-confident.

PSCC-Net - gated vs not gated (threshold 0.2)

not gated

IoU	0.33
Dice	0.41
Precision	0.46
Recall	0.48

gated

IoU	0.23
Dice	0.29
Precision	0.33
Recall	0.34

Conclusion: Detection is the bottleneck of PSSC-Net. The two heads work better independently.

PSSC-Net - other variables

Detection

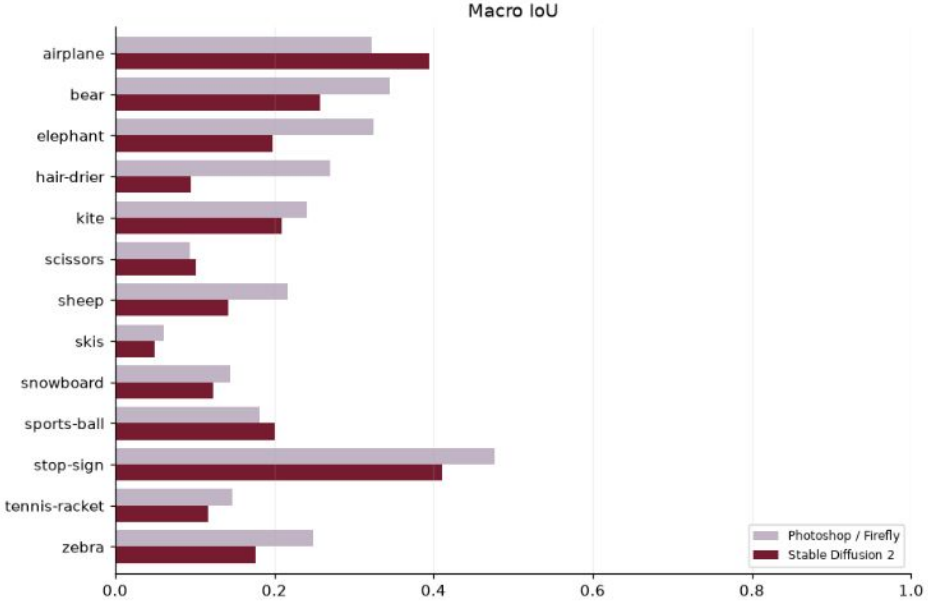
- results independent on generator method, mask type, and variant

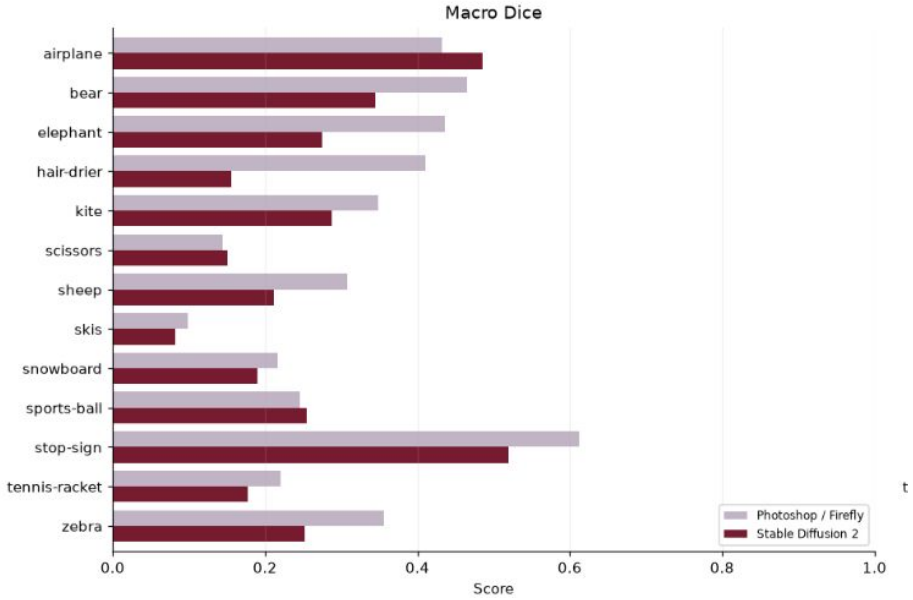
Localization

- dependent on generation method
  - PS (IoU=0.30, Dice=0.38)
  - SD2 (IoU=0.12, Dice=0.15)
- huge range for categories

category	iou_mean	dice_mean	pix_precision_mean	pix_recall_mean
skis	0.0515	0.0703	0.1539	0.0570
snowboard	0.0605	0.0835	0.1991	0.1039
tennis racket	0.0852	0.1165	0.2305	0.1344
scissors	0.1316	0.1723	0.3441	0.1495
sports ball	0.1352	0.1776	0.2859	0.1544
kite	0.1549	0.1998	0.3903	0.1907
airplane	0.2069	0.2747	0.4891	0.2676
hair drier	0.2336	0.2998	0.5810	0.2349
elephant	0.2937	0.3783	0.6756	0.3050
stop sign	0.3066	0.3786	0.5799	0.3825
sheep	0.3204	0.4023	0.6234	0.3568
bear	0.3688	0.4675	0.7270	0.3882
zebra	0.4299	0.5348	0.7901	0.4700

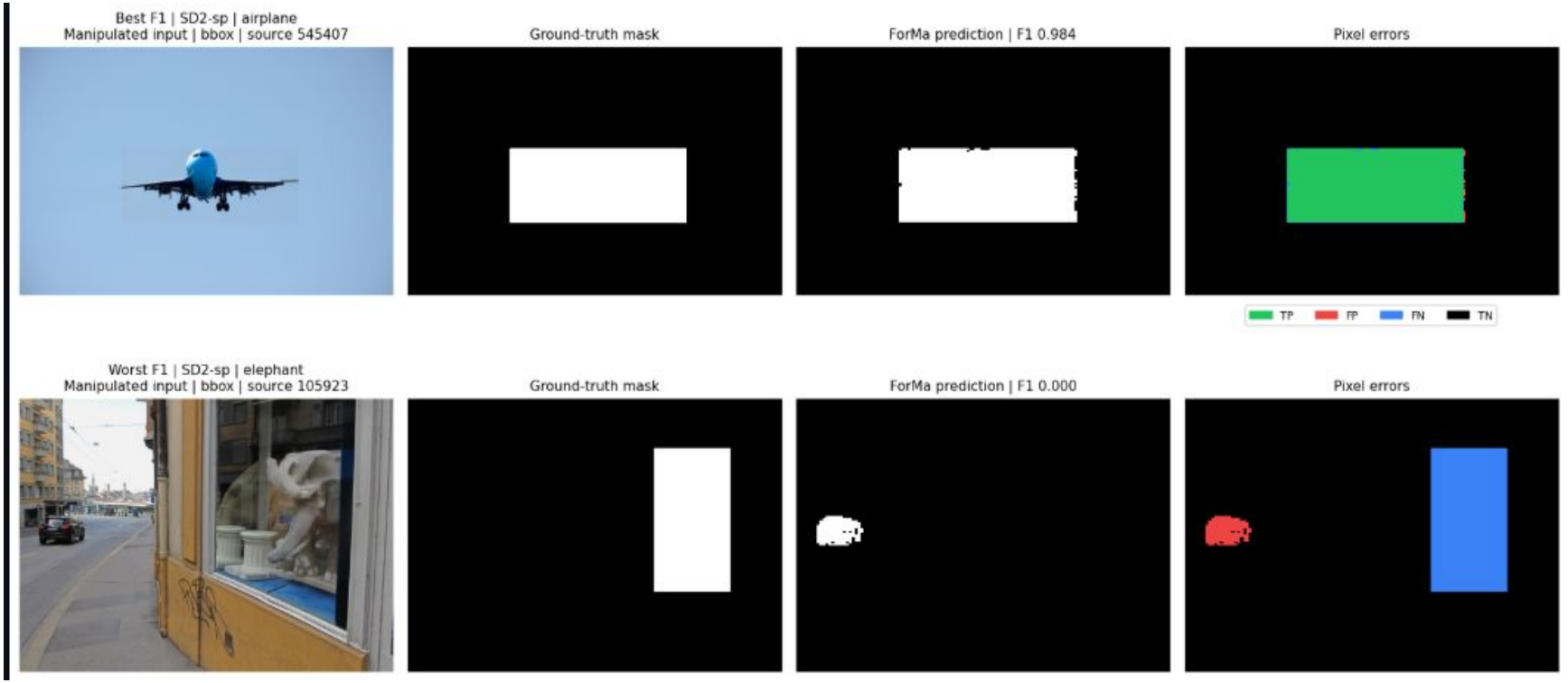
ForMa Results Interpretation



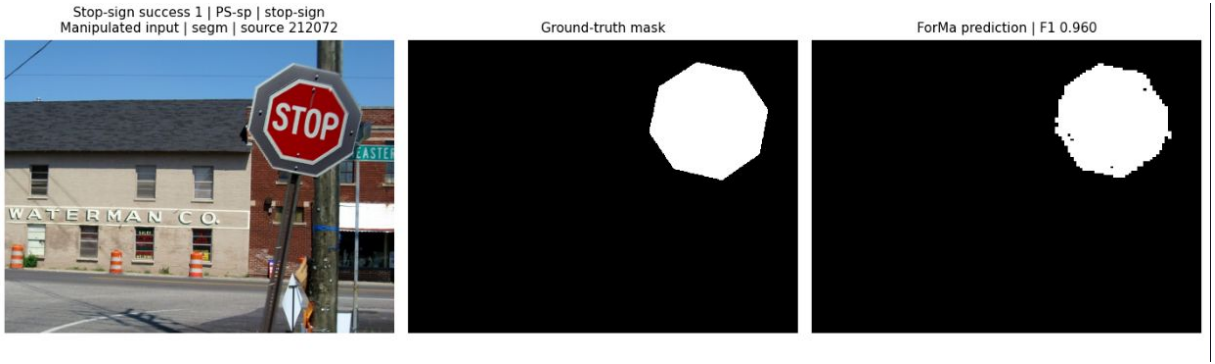


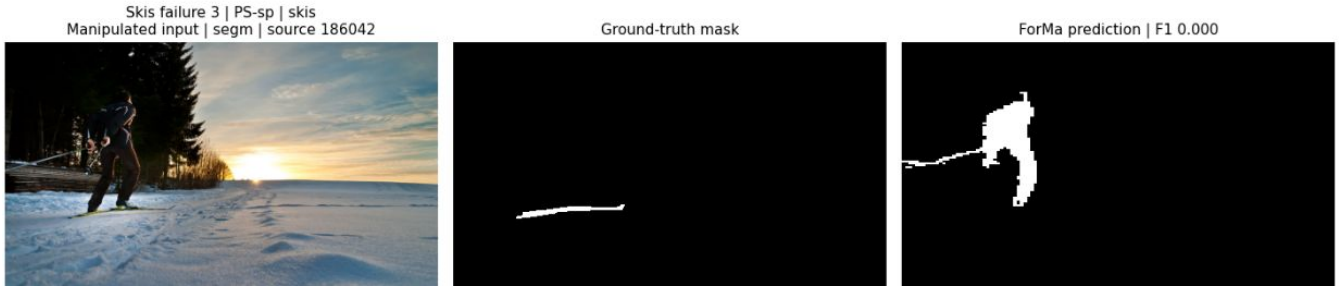
- We notice better results for Photoshop compared to stable diffusion

ForMa


Best F1 | SD2-sp | airplane
Manipulated input | bbox | source 545407
Ground-truth mask
ForMa prediction | F1 0.984
Pixel errors
TP
FP
FN
TN
Worst F1 | SD2-sp | elephant
Manipulated input | bbox | source 105923
Ground-truth mask
ForMa prediction | F1 0.000
Pixel errors

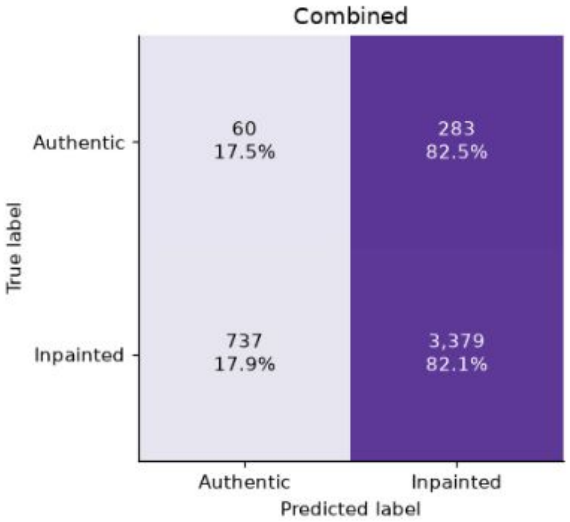
ForMa

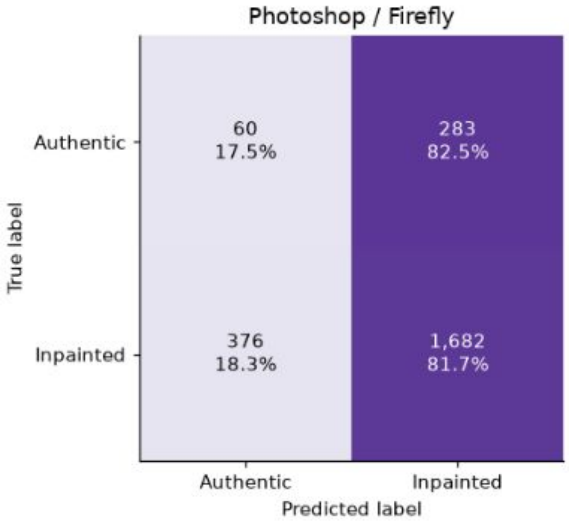


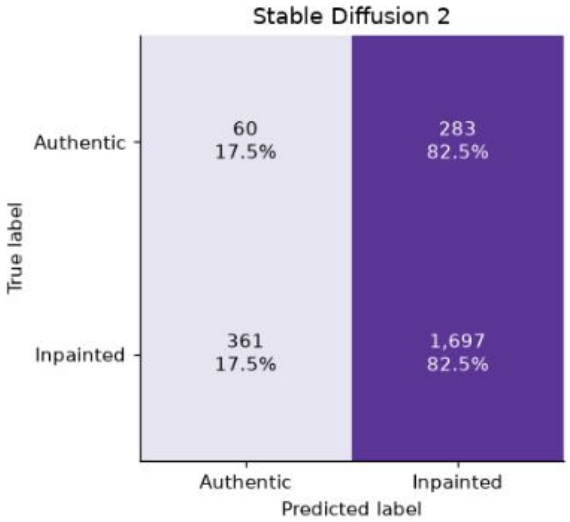


One possible explanation is the geometry of the manipulated regions: compact regions may be easier to localize, while thin, elongated regions are more sensitive to boundary errors, reducing IoU and Dice. Differences in edited-area size, background complexity, may also contribute.

ForMa



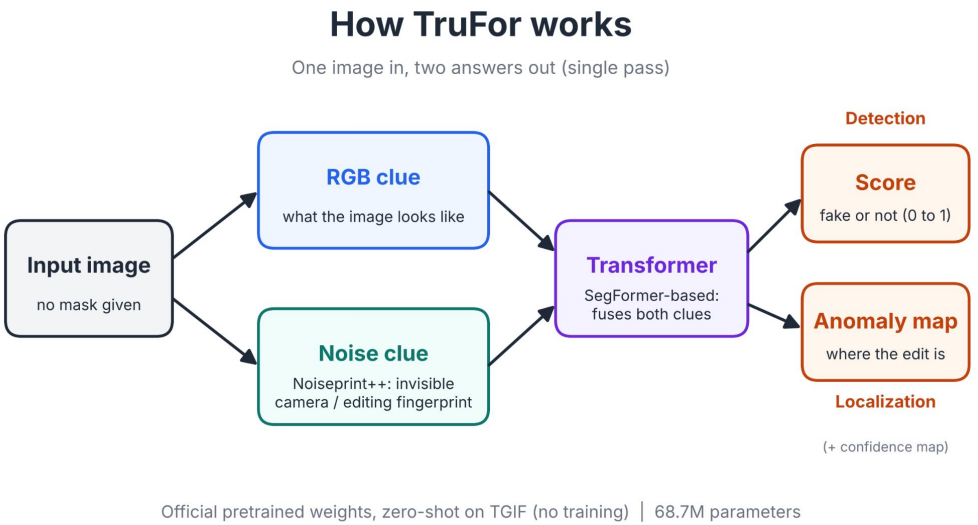




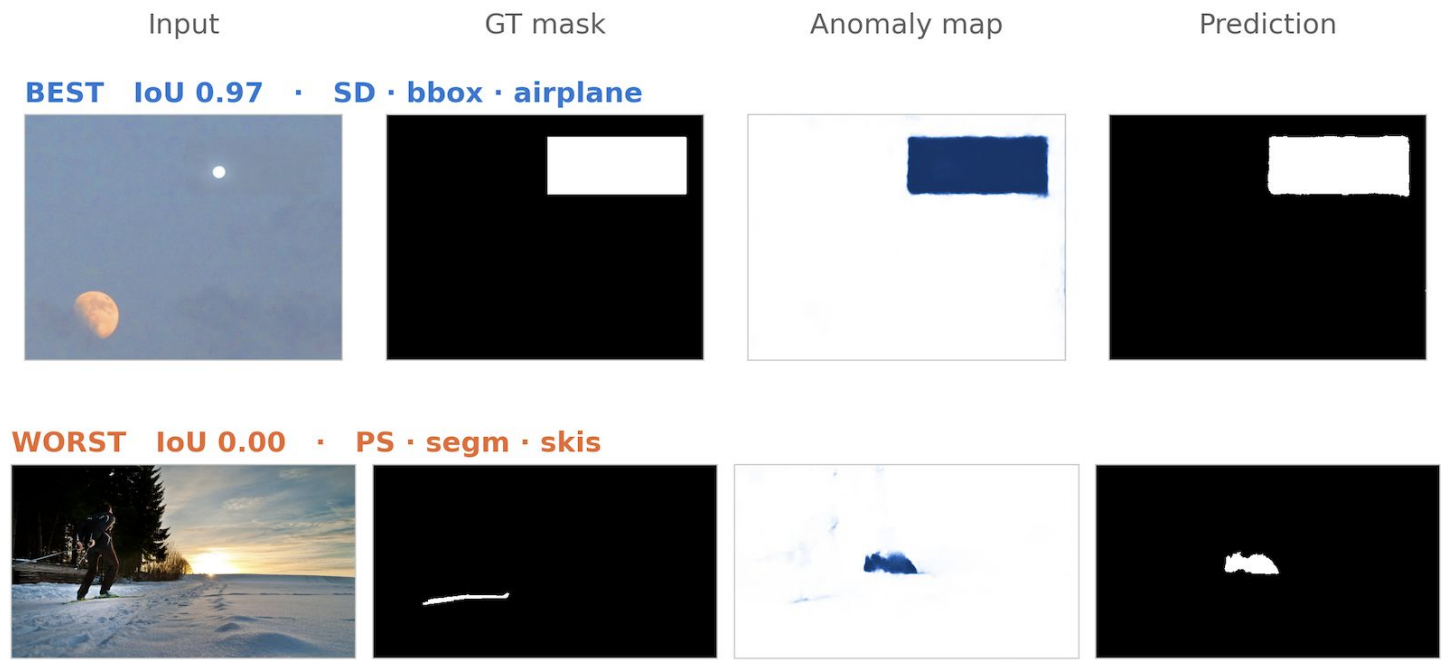
Detection with 1% threshold

TruFor: Transformer-Based Forgery Detection and Localization

- TruFor (CVPR 2023): **one image in, two answers out.**
- Image content + invisible noise "fingerprint" (Noiseprint++), combined by a **SegFormer-based transformer.**
- Outputs: integrity score (fake or not) and anomaly map (where).
- Official pretrained weights, no training on TGIF.



Localization: when TruFor works and when it fails



Detection: manipulated if score > 0.5 · Localization: map > 0.5 vs GT mask ≥ 128 · TGIF test set

Detection: what TruFor gets right, and where it fails

Input

Anomaly map

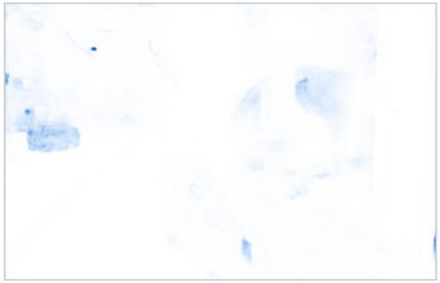
BEST: fake caught (SD): score > 0.999 → manipulated





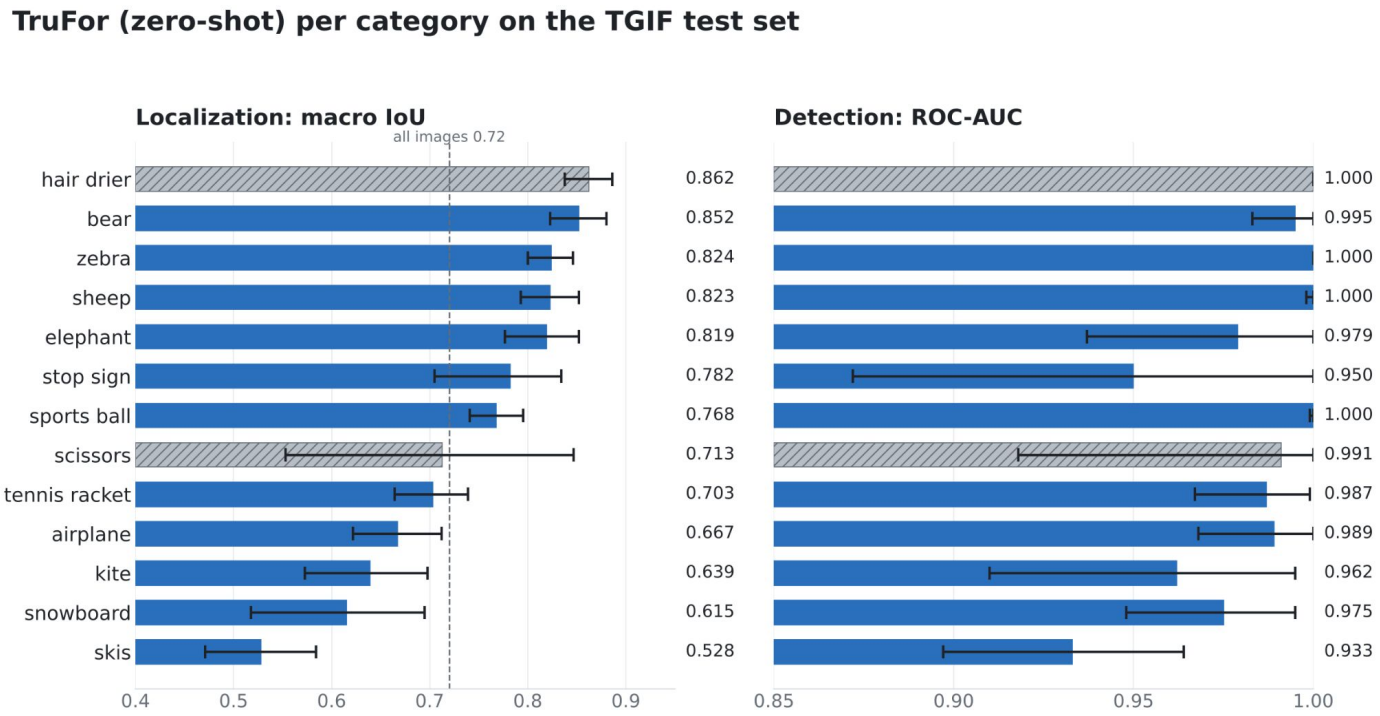
WORST: fake missed (SD): score 0.016 → authentic





Detection: manipulated if score > 0.5 · Localization: map > 0.5 vs GT mask ≥ 128 · TGIF test set

TruFor results by category



Bars: value over the 4,407 unique test images. Lines: 95% CI (bootstrap over source photos). Hatched grey = too few photos to rank (hair drier 2, scissors 8).
Note: bars start at 0.4 (IoU) and 0.85 (AUC), to make differences visible.

Challenges : reproducibility

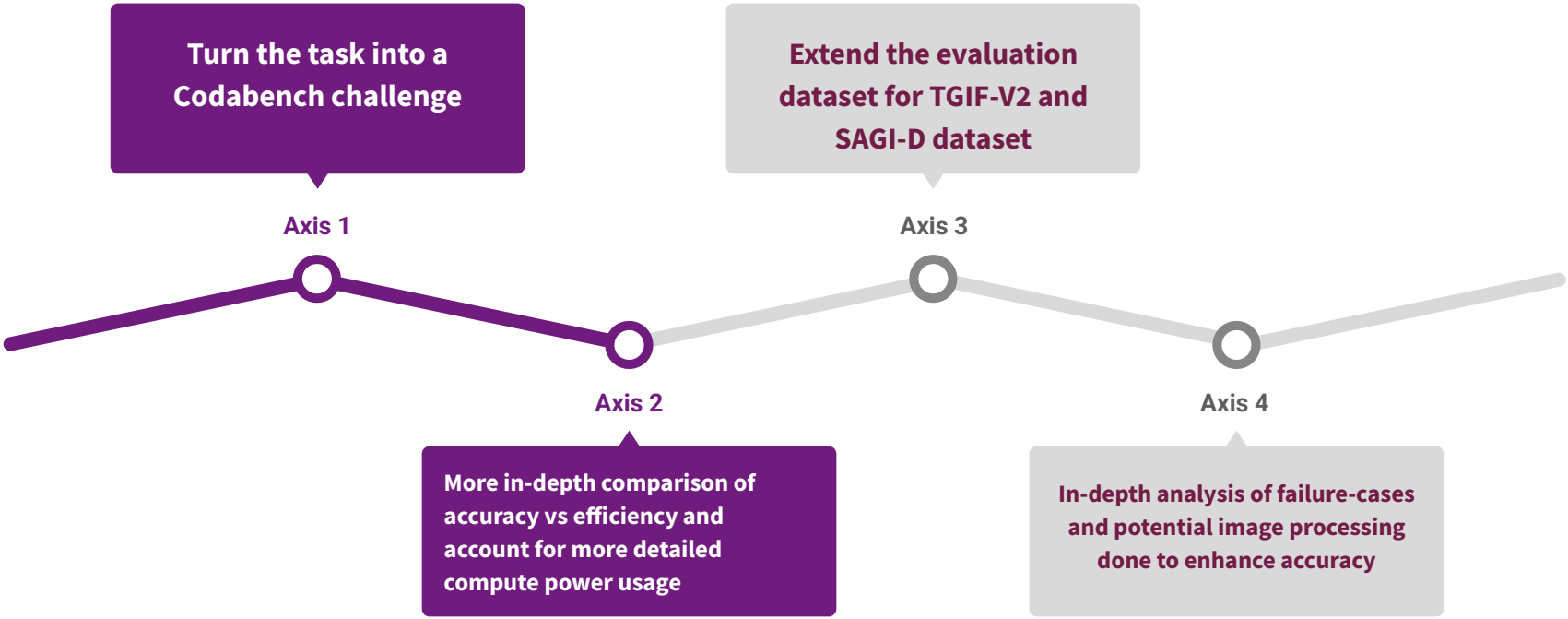
Frequency-domain model:

- **ObjectFormer: no released weights and no frequency branch in the code**, so we had to train ObjectFormer ourselves, and trained only on the test set
- **CAT-Net** — all 6 files in the authors' Drive return *"the owner hasn't given you permission"*, despite a CC-BY licence
- the frequency-domain family has **no model we could evaluate as published**

Limitations of the study

- **One dataset:** TGIF only, two inpainting sources (SD2, Photoshop). No cross-dataset generalization test, so everything we report is in-domain.
- **Limited and uneven categories:** 13 categories, very unbalanced — 38 source images for tennis racket, 7 for scissors, 2 for hair drier.
- **Limited compute power:** we had no GPU cluster and limited disk, so every model was trained or run on a reduced budget

Perspectives



Lessons learned

- Accuracy is not enough on an imbalanced set (92% fake)
- Detection ≠ localization
- Edit size and generator type strongly affect results
- Different categories and image nature affect the capacity of the detector
- A unified protocol is what makes models comparable
- Performance must be read together with model size and cost
- Various techniques exist for inpainting and it has many usages such as the technique of steganography

Recapitulation

Refresher + Setup	Results + Analysis	Perspectives
Refresher, motivation and scope, Experimental Setup	Initial Experimental Results and Interpretation and challenges	Future axes of development

AI Use Declaration

AI tools were used only to assist with coding, script adaptation, and text reformulation. All slides were created by the team.

Your questions are welcome :)

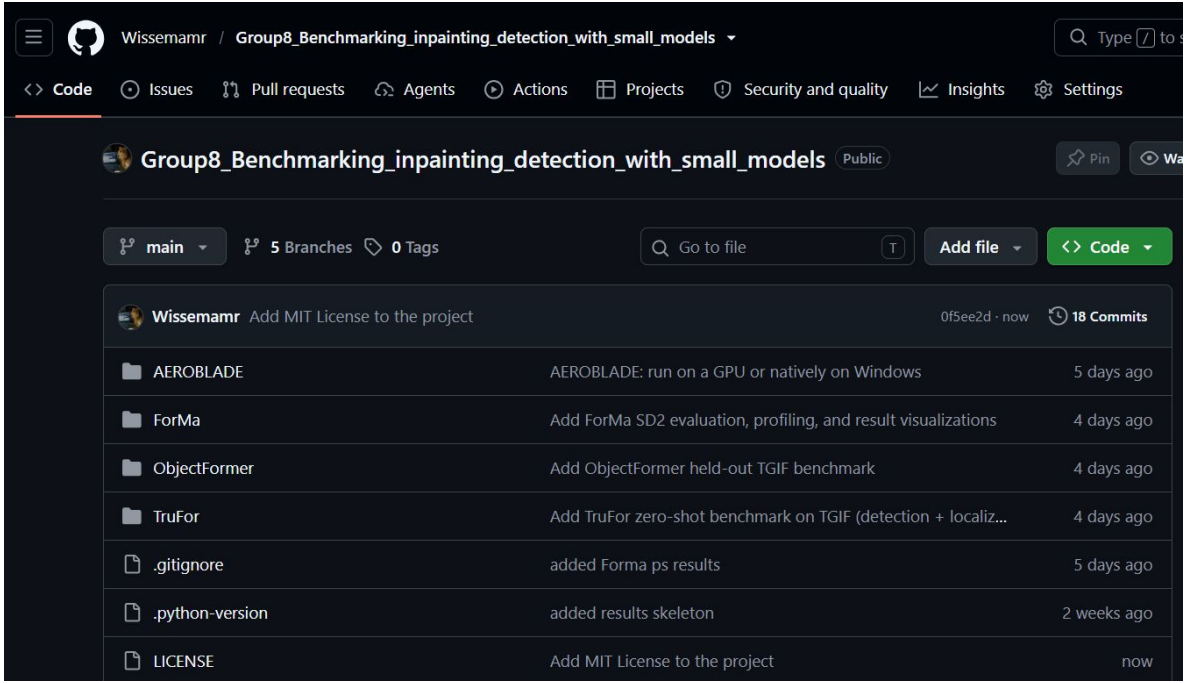
Our Team's Main Contributions



Hasti Hosseinpour	TruFor on the 3 resamples, detection + localization; per-category analysis and best/worst examples; state-of-the-art and model-categories slides
Wissem Amar	Experimenting with ForMa as an SSM model for localization and detection and reporting results, Repository organization
Dominika Chojnacka	PSCC-Net on the 3 resamples, on detection and localization, interpretation of the results, State-of-the Art of CNN-based methods, and datasets
Youssef El Otmani	AEROBLADE on the 3 resamples, full run, and on initial dataset, implementation of localisation task, analysing and interpreting results, computational comparaison run on all models, diffusion models state of the art research,
Sathya Gnanou	Frequency-domain model: trained ObjectFormer on the 3 resamples and audited its released code and weights, then tried CAT-Net.

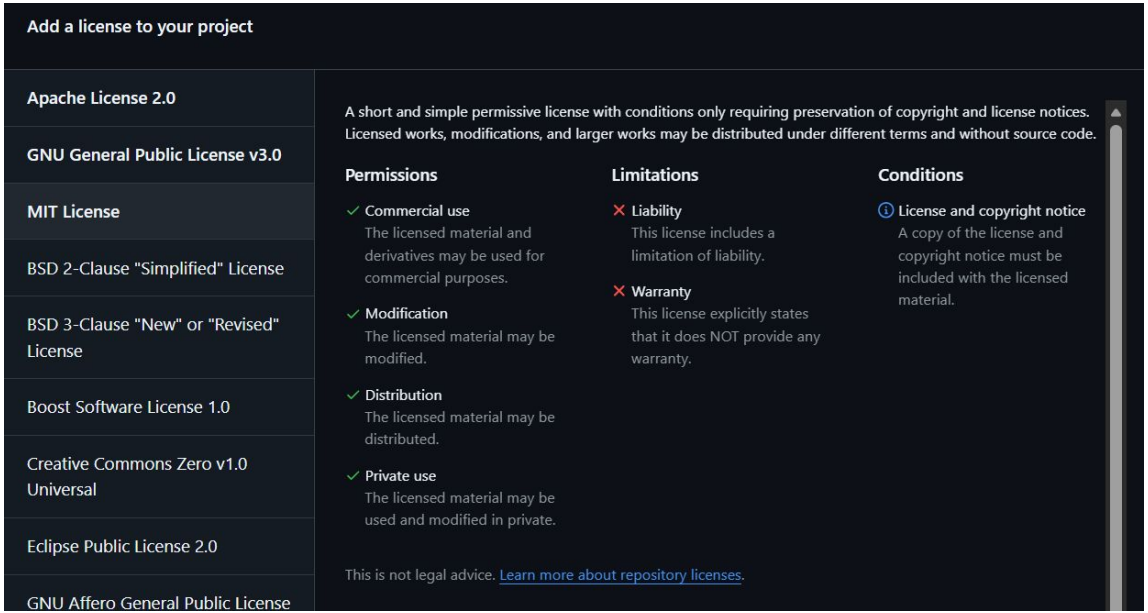
Documented Experiments Code

- The code is well documented, up-to-date and well organized
- One Folder per model experiments



Open Collaboration

- We released our code under the MIT License to make our benchmark easy to reuse, adapt, and extend
- The goal is to encourage collaboration and move towards a community leaderboard

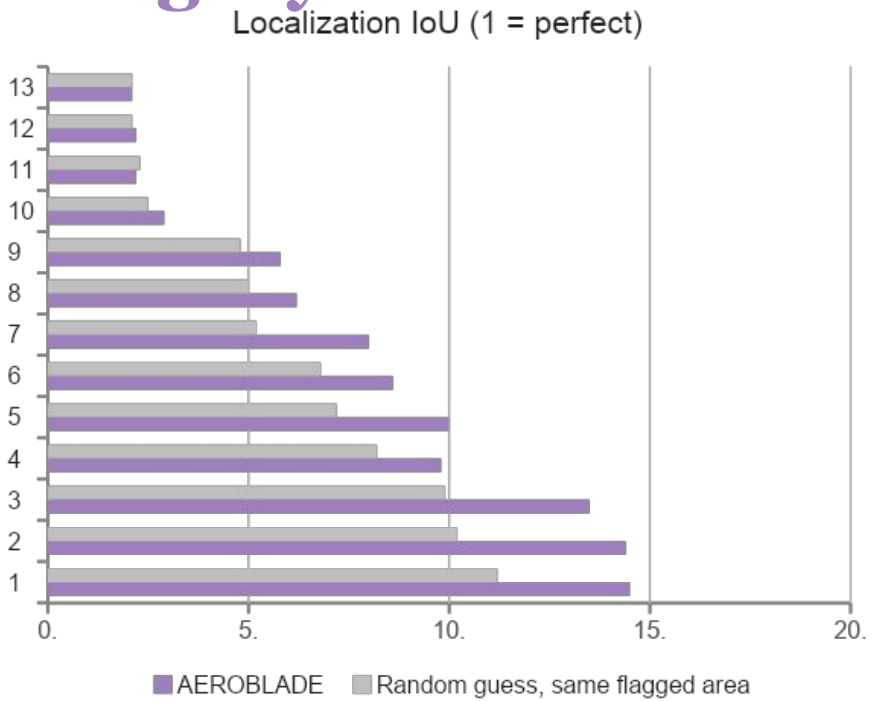


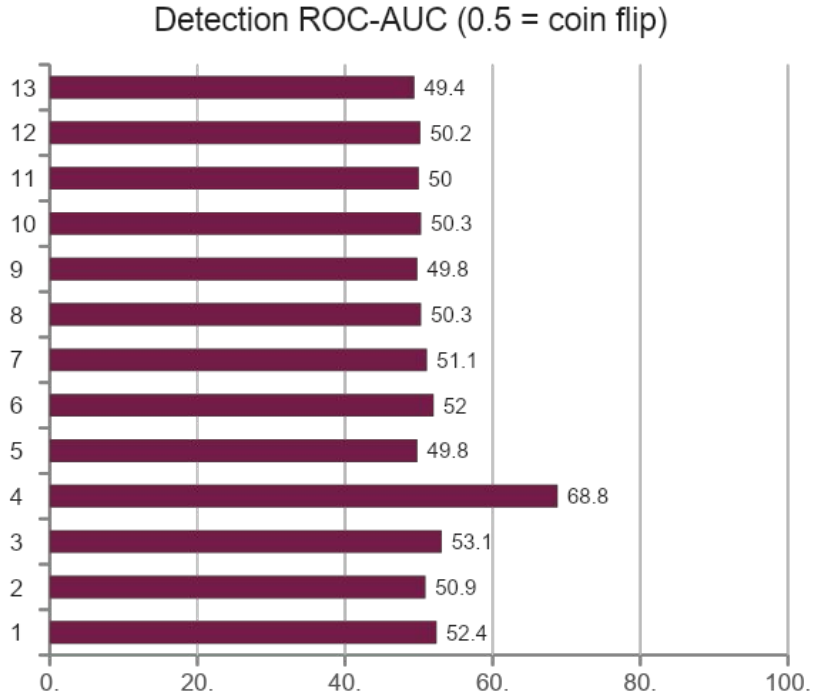
References

- M. F. Mohamed, H. A. -G. ElSayed and S. A. E. Aly, "From Reconstruction to Detection: A Comprehensive Survey on Image Inpainting Forensics," *2025 International Conference on Artificial Intelligence Science and Applications in Industry and Society (CAISAIS)*, Galala, Egypt, 2025,
- Guo, K., Cao, G., Lou, Z., Huang, X., & Liu, J. (2025). A Lightweight and Effective Image Tampering Localization Network with Vision Mamba. *ArXiv*. https://arxiv.org/abs/2502.09941
- Liu, X., Liu, Y., Chen, J., & Liu, X. (2021). PSCC-Net: Progressive Spatio-Channel Correlation Network for Image Manipulation Detection and Localization. *ArXiv*. https://arxiv.org/abs/2103.10596
- Giakoumoglou, P., & Papadopoulos, S. (2026). DiffusionPrint: Learning Generative Fingerprints for Diffusion-Based Inpainting Localization. *ArXiv*. https://arxiv.org/abs/2604.12443
- Ricker, J., Lukovnikov, D., & Fischer, A. (2024). AEROBLADE: Training-Free Detection of Latent Diffusion Images Using Autoencoder Reconstruction Error. *ArXiv*. https://arxiv.org/abs/2401.17879
- Nebioglu, E., Bilgiç, E., & Popescu, A. (2026). AI-Generated Image Detectors Overrely on Global Artifacts: Evidence from Inpainting Exchange. *ArXiv*. https://arxiv.org/abs/2602.00192
- M. -J. Kwon, I. -J. Yu, S. -H. Nam and H. -K. Lee, "CAT-Net: Compression Artifact Tracing Network for Detection and Localization of Image Splicing," *2021 IEEE Winter Conference on Applications of Computer Vision (WACV)*, Waikoloa, HI, USA, 2021

Appendix

Appendix: AEROBLADE results by object category

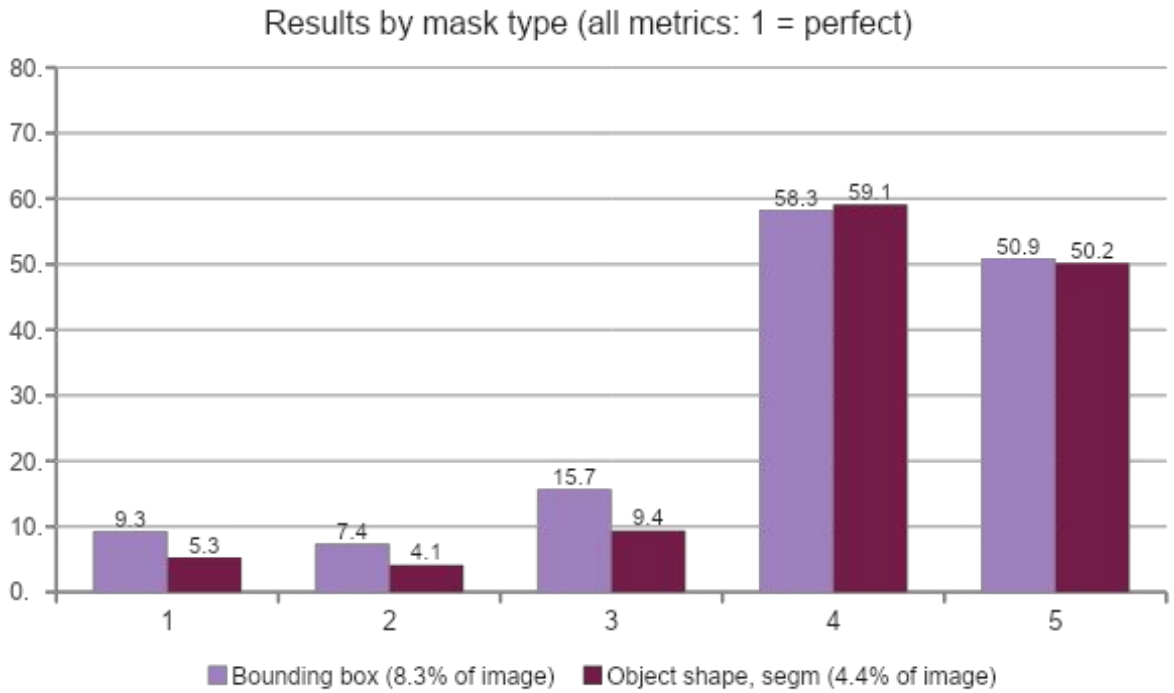




IoU follows object size, barely above a random guess · detection at chance in every category

(%) = average inpainted area. Detection: inpainted vs authentic photos of the same category; hair drier has only 2 authentic photos.

Appendix: AEROBLADE results by mask type



8.3% vs 4.4%

of the image inpainted: bbox vs segm

1.26× / 1.32×

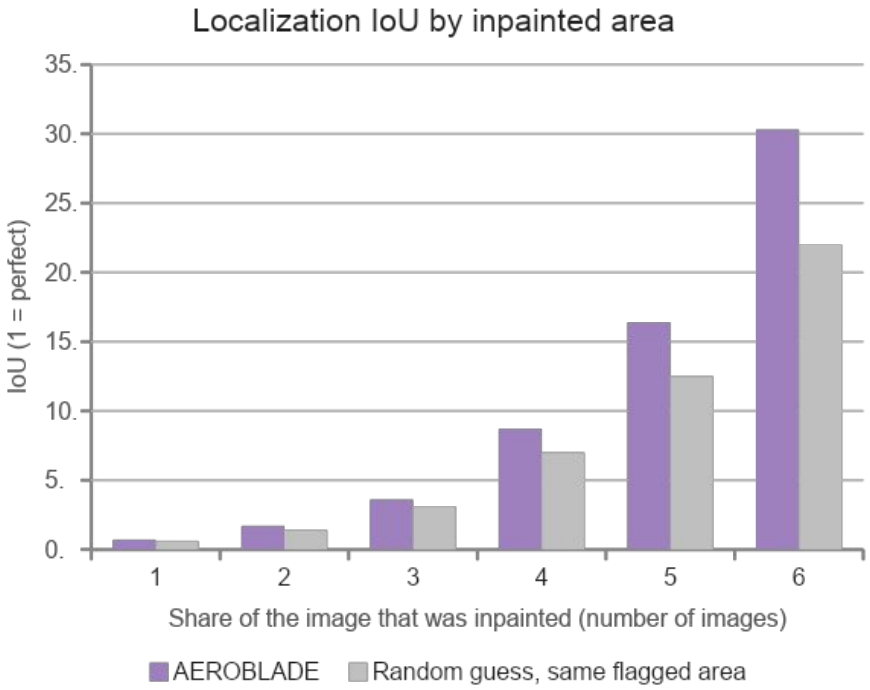
IoU over random guess: bbox / segm

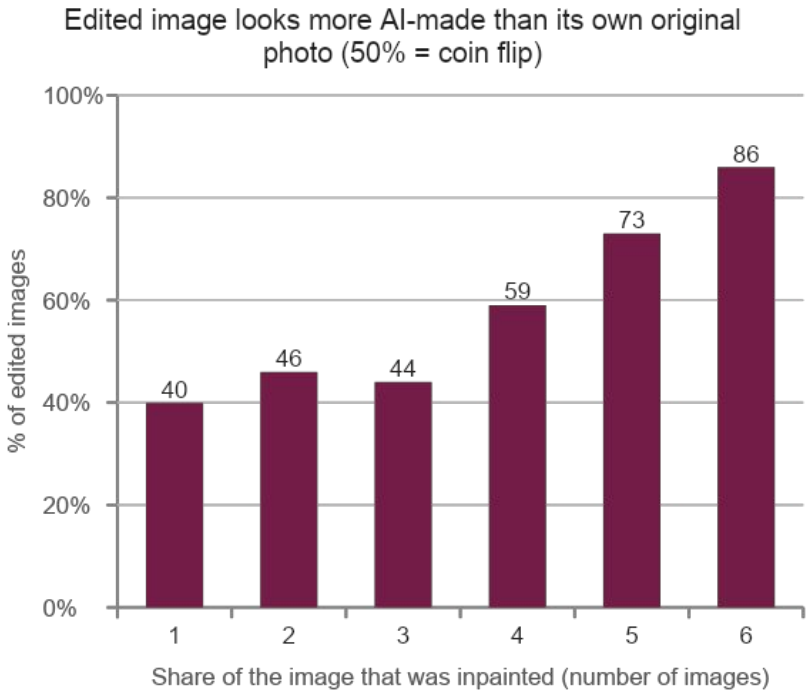
Boxes score higher only because they are bigger

Same behaviour for both mask types once the area is taken into account

Random guess: expected IoU when the same share of pixels is flagged at random. Detection: inpainted images vs all 343 authentic photos.

Appendix: AEROBLADE results by inpainted area

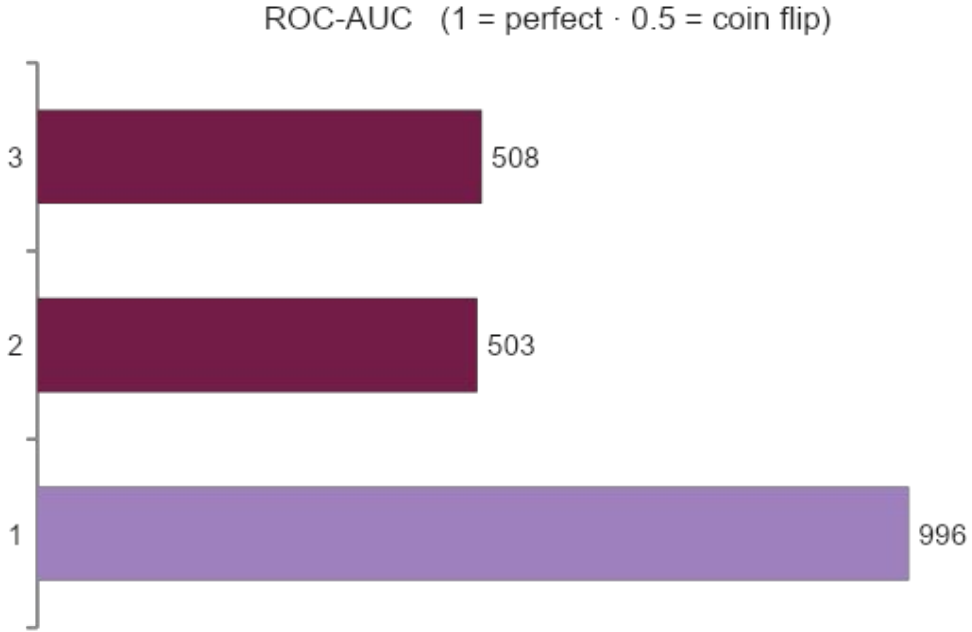




Big edits are noticed (86% for > 20%), but a 4% score shift is small next to the 47% spread between photos

(n) = number of inpainted images. Right: each inpainted image compared with its own untouched original; spread = interquartile range of authentic scores.

Detection: AEROBLADE cannot tell them apart



Typical score (median)

0.039	Authentic
0.039	SD2 inpainted
0.039	Photoshop inpainted

Same score → no threshold separates them

- Built for fully generated images: works
- Inpainting changes ~6% of the image: coin flip

appendix : AI Use Declaration



Program:

Work concerned: *(Course / Lab / Project / Submission)*

Student: **Date:**

1) Tools used *(e.g. ChatGPT, Copilot, Gemini, Claude, other)*

2) Use made *(e.g. understanding instructions, planning, rephrasing, generating a code example, debugging, testing)*

3) Significant prompts (summary or excerpts)

4) AI outputs actually used

5) Personal verifications and corrections

6) Personal contribution

Data, Data, Data

"It is a capital mistake to theorize before one has data capital mistake to theorize before one has data"

- Sir Arthur Conan Doyle

Careful and backed selection of data is crucial

Dataset: TGIF

Text-Guided Inpainting Forgery

- around 80k images (in total)
  - ~3000 authentic, ~75000 manipulated
- manipulated using 3 different methods
- originate from MS-COCO
- supports localization

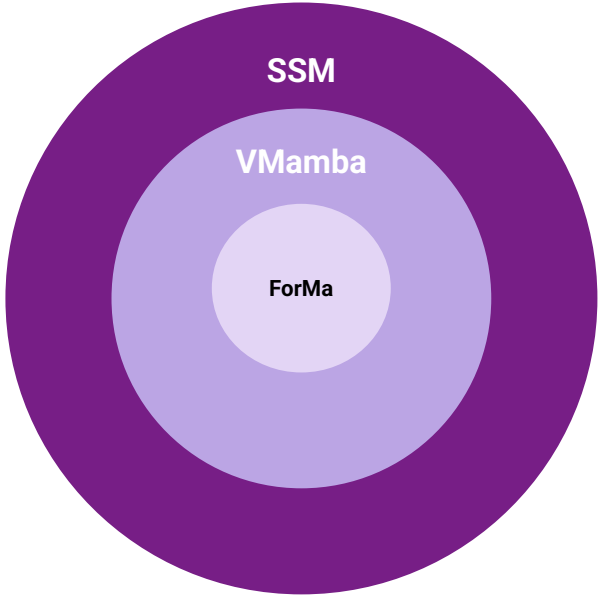
This was before

And what have we been up to for the past week?

Experimental Setup
Example : ForMa (Guo et al., 2025)

Goal

Evaluate the pretrained ForMa model for **inpainting/tampering localization** under a common benchmark.



Experimental Setup / ForMa (Guo et al., 2025)

Forensic Vision Mamba: Lightweight SSM architecture
Task: Pixel-level tampering localization

Model Overview

Task Setting: Zero shot evaluation

Target Dataset : TGIF Benchmark

Input Resolution : 512 x 512

Execution Protocol

Experimental Setup / ForMa (Guo et al., 2025)



TGIF Image

Original Sample

Preprocess

Resize 512 x 512

ForMa Mamba

2 channel output map

Post Process

Softmax -> Argmax -> Binary Mask

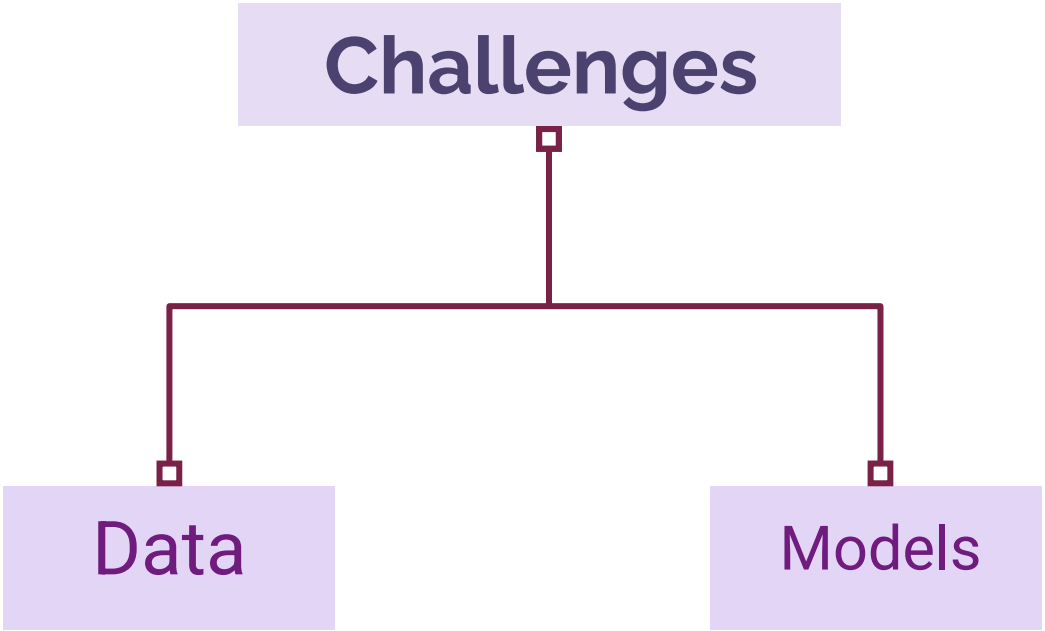
Comparison between the mask and the inpainted image
Performance : IoU • Dice • Precision • Recall • F1
Efficiency : Parameters Count and Model size, FLOPS, RAM footprint

Experimental Setup / ForMa (Guo et al., 2025)

Initial Environment

```
Python3.10
PyTorch1.13.1
DeviceCPU Only
ProcessorIntel i7 11th Gen
uv for dependency management
```

Encountered Challenges so far



Challenge 1: Data duplicates

Problem: Three types of the same image in the authentic dataset







Solution: Keeping two sizes, and comparing their results

Challenge 2: Data leakage

Problem: TGIF dataset contains some images from MS COCO, on which ForMa was trained

Solution: Using metadata, removing all the instances from TGIF that has been seen by ForMa in the training phase

Challenge 3: Class imbalance

Problem: Approximately 4 times more forged images than real ones. After removing duplicates, 6 times more.






Solution: Use metrics that ignore class ratio, or possibly for randomly choose forged images to balance the data.

Challenge 4: Detection vs Localization

Problem: Localizing inpainings even when detection head marked an image as authentic. It increases the localization metrics.

Solution: Introduce a gate that only localizes inpaintings when the detection head marked it as a forged image

Challenge 5: Adapting SegFormer-B1 to Inpainting Forensics

Problem

SegFormer-B1 was not pretrained for image forensics

- 13.7M-parameter lightweight Transformer
- Pretrained on ImageNet-1K and fine-tuned on ADE20K semantic segmentation
- **No prior knowledge of inpainting/manipulation traces**

Solution??

TGIF SD2-sp exploratory experiment

- Replaced the original **150-class ADE20K head** with a binary localization head
- Leakage-free split by original COCO image
- **1,918 train / 238 validation / 245 test samples**

Epoch	Train loss	Val loss
1	0.9241	0.8249
2	0.5821	0.7655
3	0.4509	0.7529

our new solution TruFor

Why TruFor?

TruFor (Trustworthy Forensics) is specifically designed for **image manipulation detection AND localization**.

Why we added it

- Already trained for forgery.
- Same model family as ours.
- Does both tasks natively.

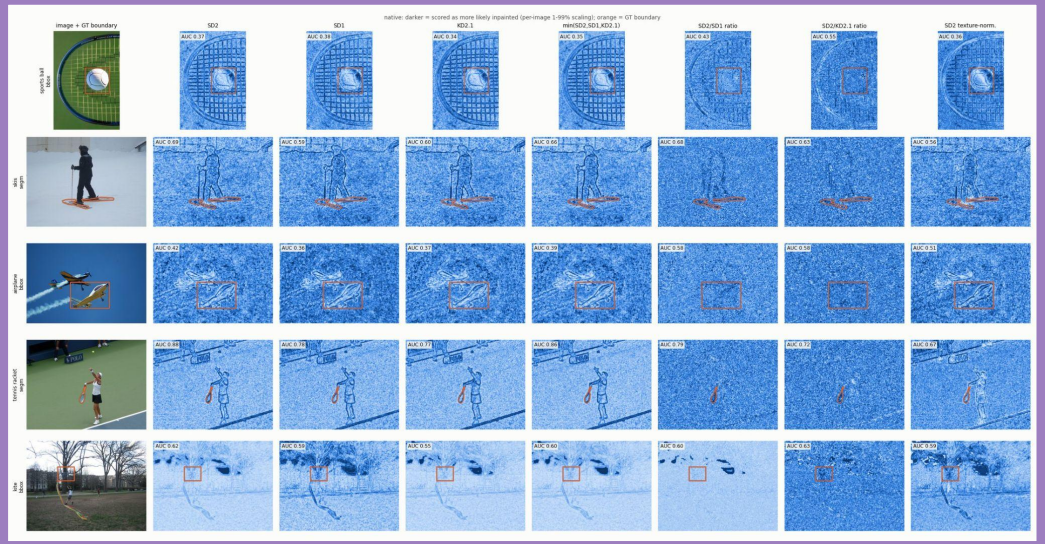
≈68.7M parameters | pretrained forensic weights | no task-specific training required

Our current checkpoint

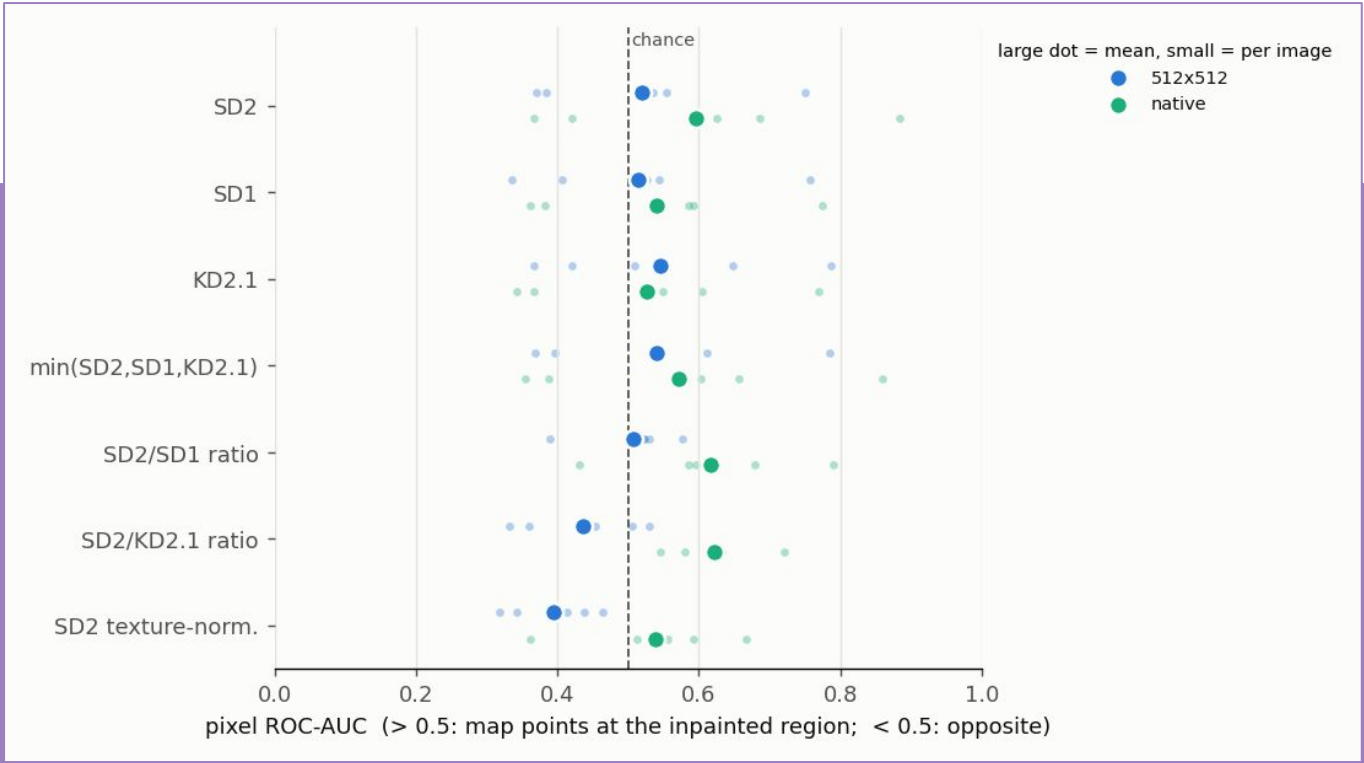


Preliminary AEROBLADE Results
Complex images vs Localization

Problem: Aeroblade heatmap light up on textured/complex areas rather than the actual inpainted region.



Challenge 7: Complex images vs Localization



This shows initial limitation of AEROBLADE on localisation task

Preliminary PSCC-Net Results

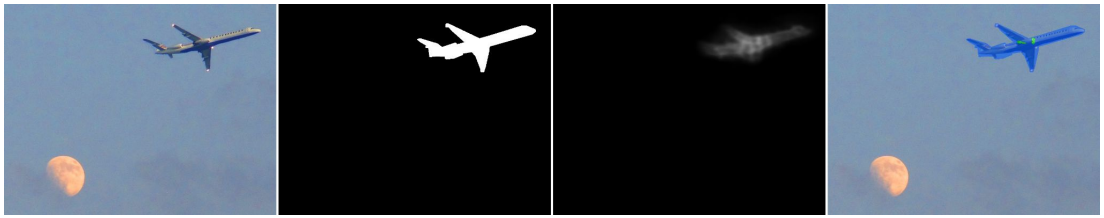
Run PSCC-Net model on a small subset of the data (not taking in the consideration the limitations of the data)

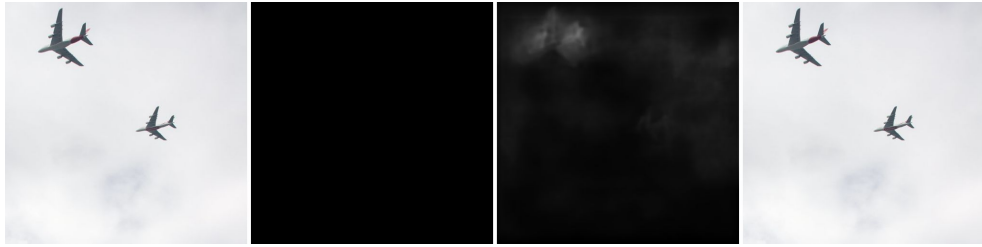
Results (326 forged images (PS, SD2), 84 authentic, airplane images)

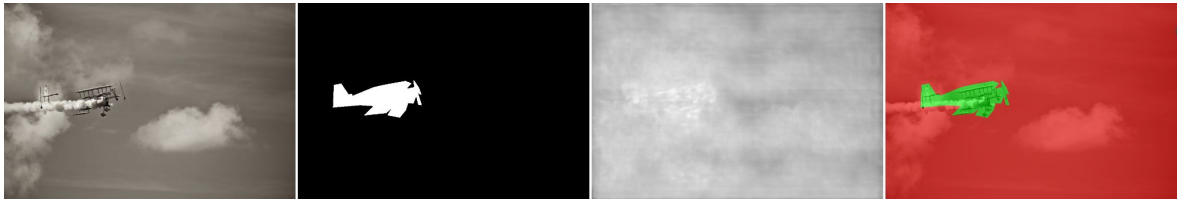
- AUC: 0.62 (slightly above chance)
- Recall: 0.71 → we catch a lot of fake
- 49% of real photos are flagged as fake
- PS is easier than SD2

Visual Examples

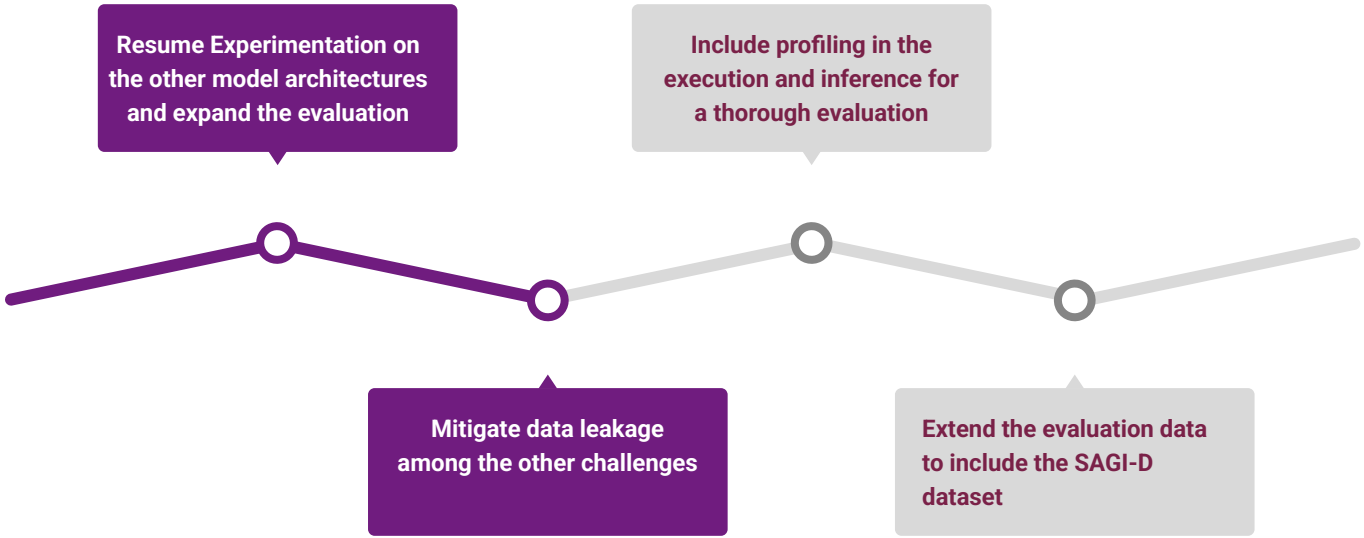




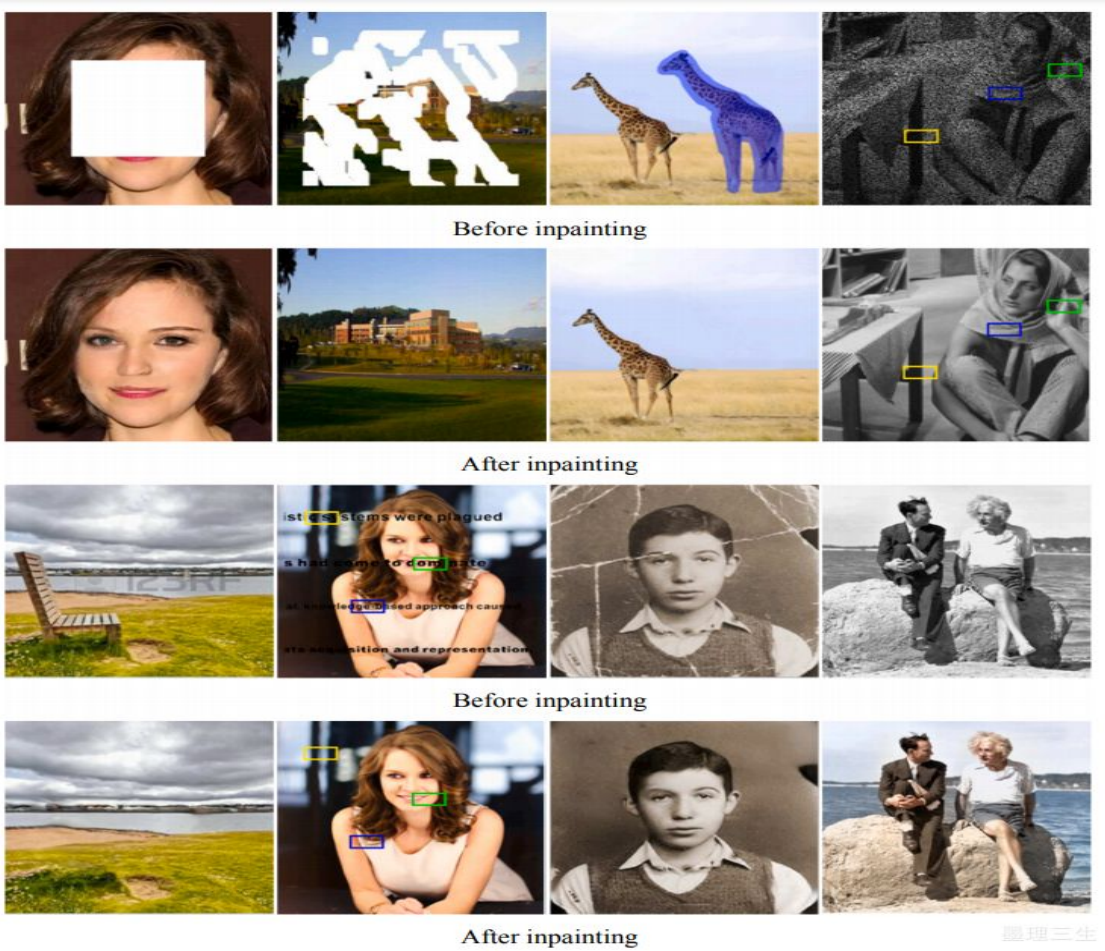




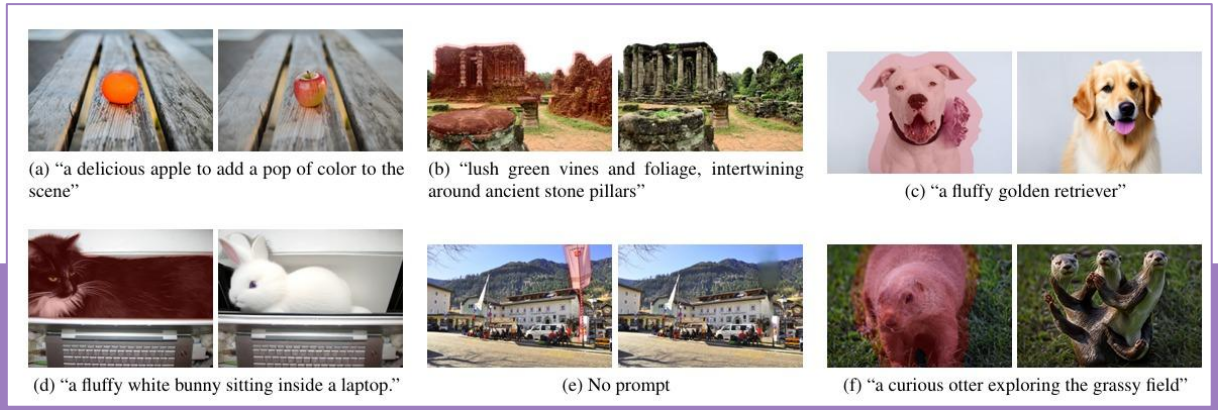
Next action items



Examples of image inpaintings:



Dataset: SAGI-D



- Largest AI-inpainting dataset to date
  - ~95,800 inpainted images (MS-COCO, RAISE, OpenImages),
  - built over multiple SOTA inpainting models
- original image, inpainting mask, and prompt → supports localization
- tests for generalization

Challenge 4: Masks

Problem: Can we compare bbox mask with fitted masks ?



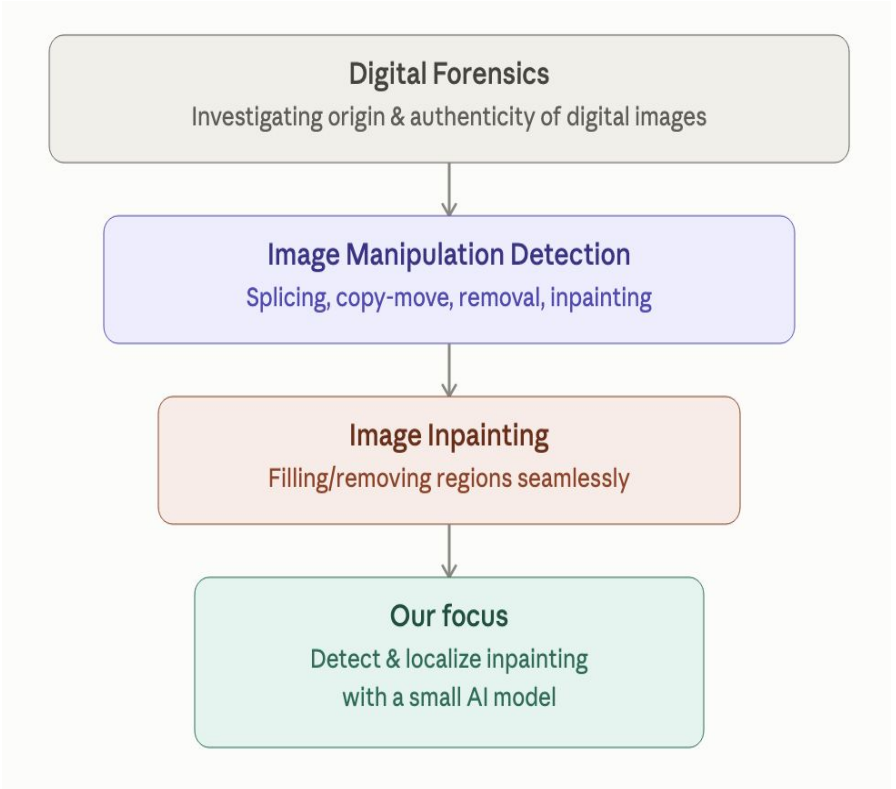
Solution: Separate result analysis

What is inpainting?

What it is:
filling missing/removed pixels so the result is visually indistinguishable from an untouched photo

Why it exists:
Legitimate: restoring damaged old photos/paintings, removing unwanted objects

Why it matters:
undetectable edits can fabricate evidence, spread misinformation, or manipulate what looks like a real photo



What is the best small AI model for inpainting-based synthetic image detection?

Presentation n°2 : Methodology

21/09/2026

Pr Adrian Popescu

Wissam AMAR • Hasti HOSSEINPOUR • Dominika CHOJNACKA • Youssef EL OTMANI • Sathya GNANOU

CNN-based: PSCC-Net

Progressive Spatio-Channel Correlation Network

Why?

- performs especially well with localization compared to other CNN baselines
- easy implementation: baseline

Objectives

Following the **SMART objectives Framework**

Specific: compare 5 selected models, small, inpainting detection and localisation

Measurable: Specific evaluation metrics for detection and localisation

Achievable: a detailed comparative study on a common benchmark

Relevant: only inpainting and only small models and only benchmarking

Time-bound: in the span of 1 week

CNN-based: PSCC-Net

Progressive Spatio-Channel Correlation Network

Why?

- performs especially well with localization compared to other CNN baselines
- easy implementation: baseline

PSCC-Net: Architecture

Top-down path:

- resolutions in parallel
- cross-scale information sharing

Bottom-up path:
Localization Detection

SCCM → Spatio-Channel Correlation Module

- spatial correlation → compares regions
- channel-wise correlation (edges, colors, etc.)

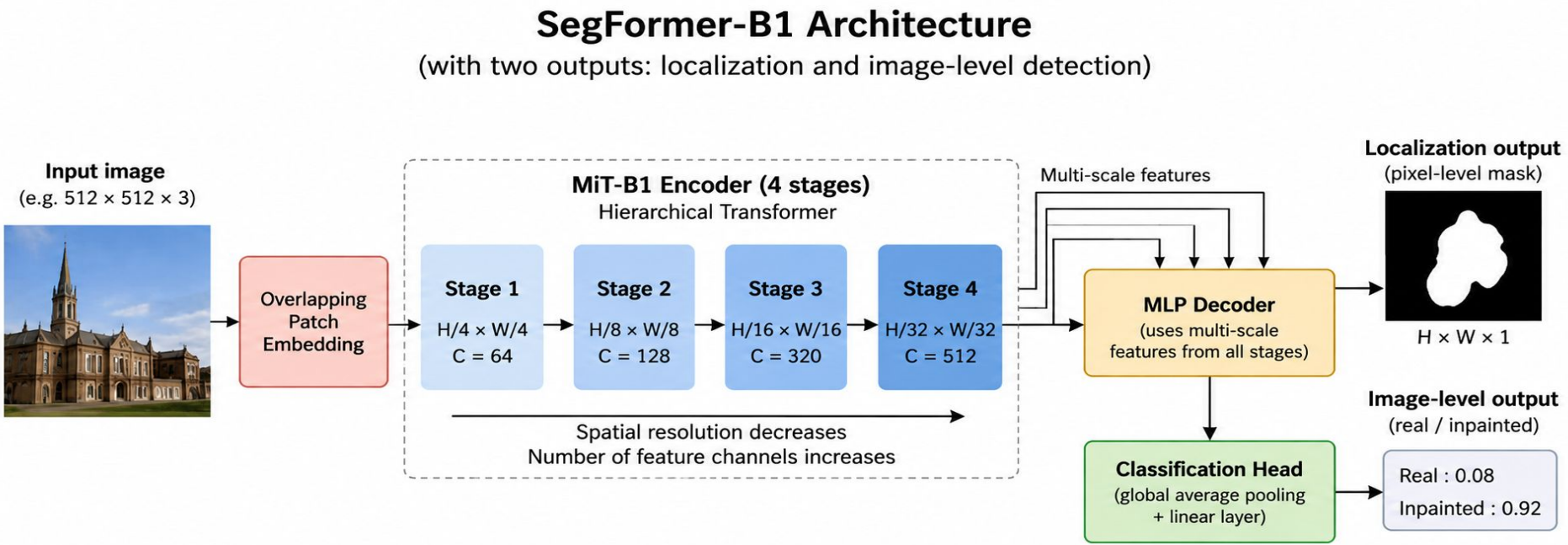
separate detection head

Transformer Candidate: SegFormer-B1

Why SegFormer-B1?

- **Localization by design**: native pixel-level segmentation architecture.
- **Multi-scale features** :our hierarchical stages capture information at different spatial scales.
- **Relatively lightweight** : ~13.7M parameters, much smaller than many large Vision Transformers.
- **Forensic relevance** : SegFormer-B1 has recently been used as the local feature module in **VAAS (2026)** for image-manipulation localization.

How does SegFormer-B1 work?



Reconstruction-based candidate : AEROBLADE

How it works

Ricker, Lukovnikov & Fischer, 2024

Training-free detector for latent-diffusion images using Autoencoder Reconstruction Error. Encode→decode a suspect image once through a pretrained VAE, then measure the change with the LPIPS perceptual distance.

One quick pass, no training.
An AE pool (SD1, SD2, Kandinsky) covers an unknown generator

Why we chose it

fits our compute axis

Latent diffusion is today's dominant inpainting, so coverage is high while cost stays minimal.

limitation: blind to GAN, classical & non-AE fills

One pass, not fifty

Other diffusion-based detectors rebuild the image to test it (add noise, then run the generator's dozens of times). **AEROBLADE removes that loop.**

	Diffusion-based (DIRE, DiffForensics)	AEROBLADE
How it rebuilds	noise → denoise loop	**encode → decode**
Passes / image	~50 (U-Net)	**1 (autoencoder)**
Training	trains a classifier	**none**

Model actually run per image (parameters)



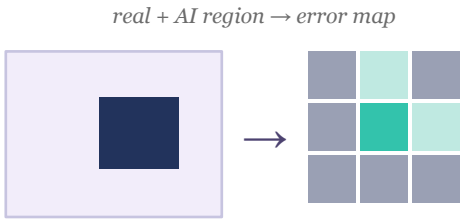
AEROBLADE borrows only the autoencoder of the same SD model — the 860M U-Net is never loaded or run (plus a small fixed LPIPS network).

From a score to a map

1. **Drop the averaging.** LPIPS normally averages over the image; skip it and the error becomes a per-pixel map.

2. **The signal inverts for inpainting.** The AI-filled region rebuilds too well, so it lights up as a low-error island against the real background.

3. **The authors stop at "qualitative".** They show it works but leave predicting the precise inpainted region as future work.



Our contribution

Turn the map into a thresholded mask and benchmark AEROBLADE as a training-free localizer
and test whether this local cue survives background restoration (the global-artifact critique of Nebioglu & Popescu).

Frequency-Domain: ObjectFormer

- A **DCT-based frequency-domain** model
- Does **detection and localization** in one jointly trained model
- EfficientNet-b4 backbone, ≈ **19M parameters**

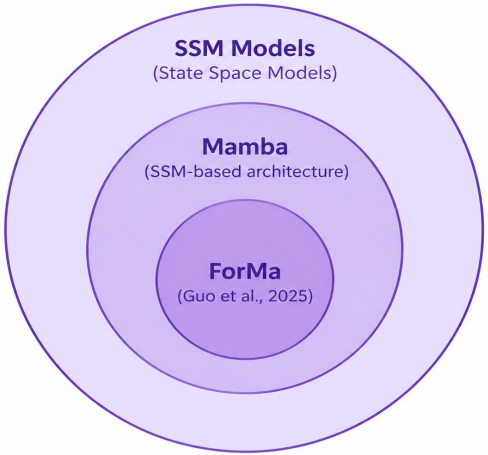
Why?

- **6× smaller** than EWSN (~130M) and CAT-Net (114M)
- One of the few small models doing **both tasks**
- Ablation: the frequency branch is worth of an accuracy of **+12.9 AUC**

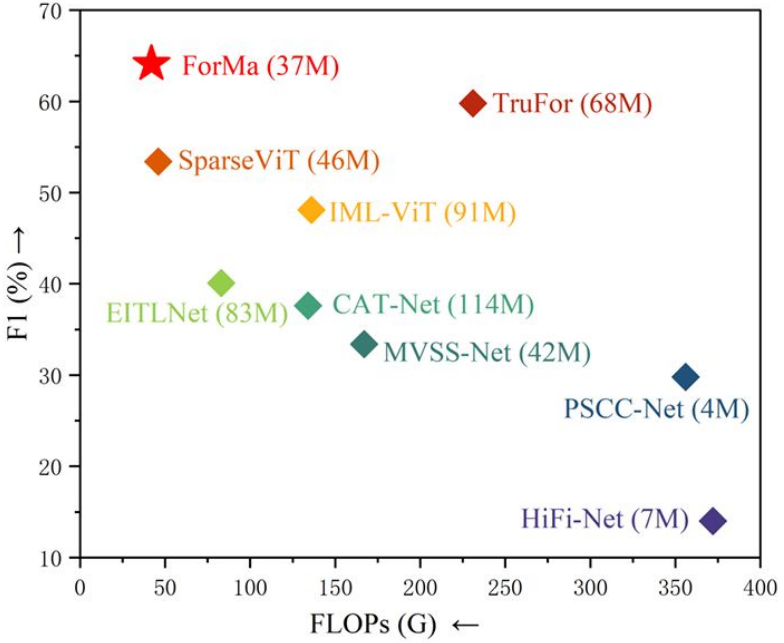
ObjectFormer: How it works?

- DCT → high-pass filter → back to image space
- High-frequency features fused with RGB features
- 16 object prototypes check object consistency
- Two heads: pooled score (detection) + upsampled mask (localization)

FORensic network based on vision MAmba (ForMa) (Guo et al., 2025)



Balance between long-range dependencies, strong feature representation capabilities and linear complexity w.r.t input size



Comparison of average F1 across 10 standard benchmark datasets and FLOPs (calculated in 512 × 512 input size) with model parameters. Our method achieves the best F1 (64.1%) with the lowest FLOPs (42G) and parameters (37M).

FORensic network based on vision MAmba (ForMa) (Guo et al., 2025)

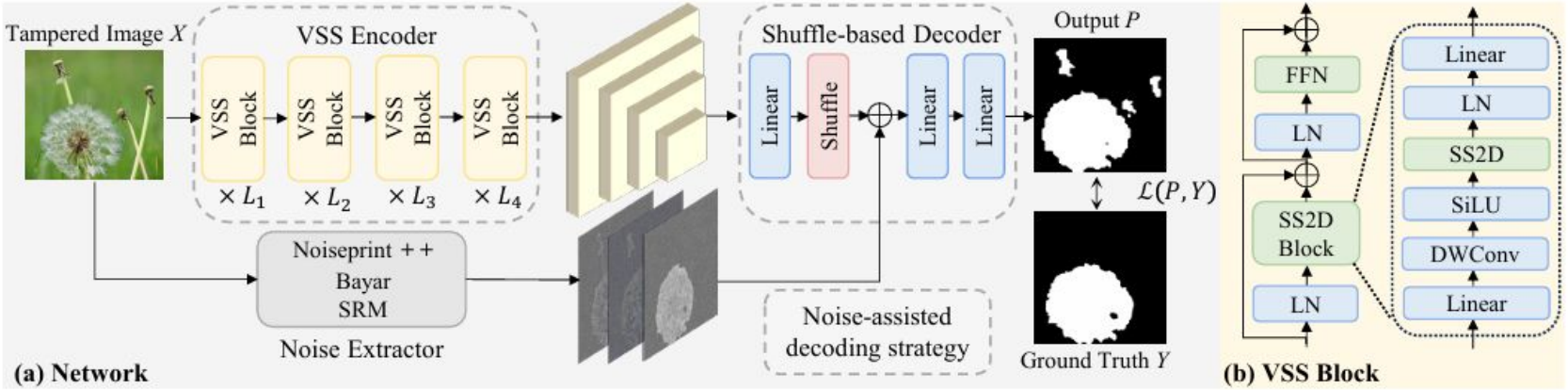


Fig. 2. (a) Architecture of proposed ForMa. L_i={2, 2, 9, 2}. Linear, Conv, and Shuffle refers to the linear, convolution and pixel shuffle layers, respectively. $\oplus$ represents element-wise addition. (b) Structure of the VSS Block. It includes a depthwise convolutional layar (DWConv), SiLU activation function, SS2D module, and linear normalization (LN). The VSS Block, Shuffle-based Decoder, and Noise Extractor are learnable.

FORensic network based on vision MAmba (ForMa) (Guo et al., 2025)

Dataset	Image Count	F1 (%)	IoU (%)
Columbia	160	94.9	93.9
DSO	100	38.7	28.3
CASIAv1	920	67.3	72.9
NIST	564	45.4	38.5
Coverage	100	50.9	58.7

Dataset	Image Count	F1 (%)	IoU (%)
Korus	220	23.5	30.4
Wild	201	57.3	48.7
CoCoGlide	512	45.3	36.2
MISD	227	70.3	57.5
FF++	1,000	81.9	71.8
Average (All 10)	**4,004**	**64.1**	**56.2**

*Code available for reproducibility

Comparison

Model	Category	Params	Memory / Hardware	Localization	Metrics
PSCC-Net	CNN-based	3.6	Single consumer GPU	Yes	F1 score 5 benchmarks
ObjectFormer	Frequency Domain	19M	GPU	Yes loc + det both native	F1 = 89.7 %
SegFormer-B1	Transformer	13.7M	GPU	Yes pixel-level segmentation	Standalone forensic F1: N/A VAAS (B1-based): F1 = 94.1%
AEROBLADE	Reconstruction-based	~ 67M/ 84M (depending on the AE pool)	Single consumer GPU	no, but per-pixel heatmap	used for detection, localisation marked as future work on the paper
ForMa	SSM	37M	Single GPU	detection + localisation	F1 = 64.1% IoU = 56.2% and 10 tampering datasets

Evaluation Metrics

Detection

- Accuracy
- Confusion Matrix
- F1 score
- ROC-AUC

Localization

- IoU
- Dice Score
- Pixel-wise Recall
- Pixel-wise Precision

Efficiency Metrics : Compute power, Model Size, trade-off

Recapitulation and Q&A

Context, Gap, RQ and Objectives

Candidate Models and Comparison

Evaluation Metrics and Datasets

Questions time!

Evaluation Metrics

Visualizing Localisation Metrics



Ground Truth Mask (G)

$|G| = 100$ pixels



Predicted Mask (P)

$|P| = 120$ pixels



Intersection ($P \cap G$)

$|P \cap G| = 80$ pixels



Union ($P \cup G$)

$|P \cup G| = 140$ pixels

IoU

$$\frac{80}{140} = 0.571$$

(57.1%)

Dice Score

$$\frac{2 \times 80}{100 + 120} = 0.727$$

(72.7 %)

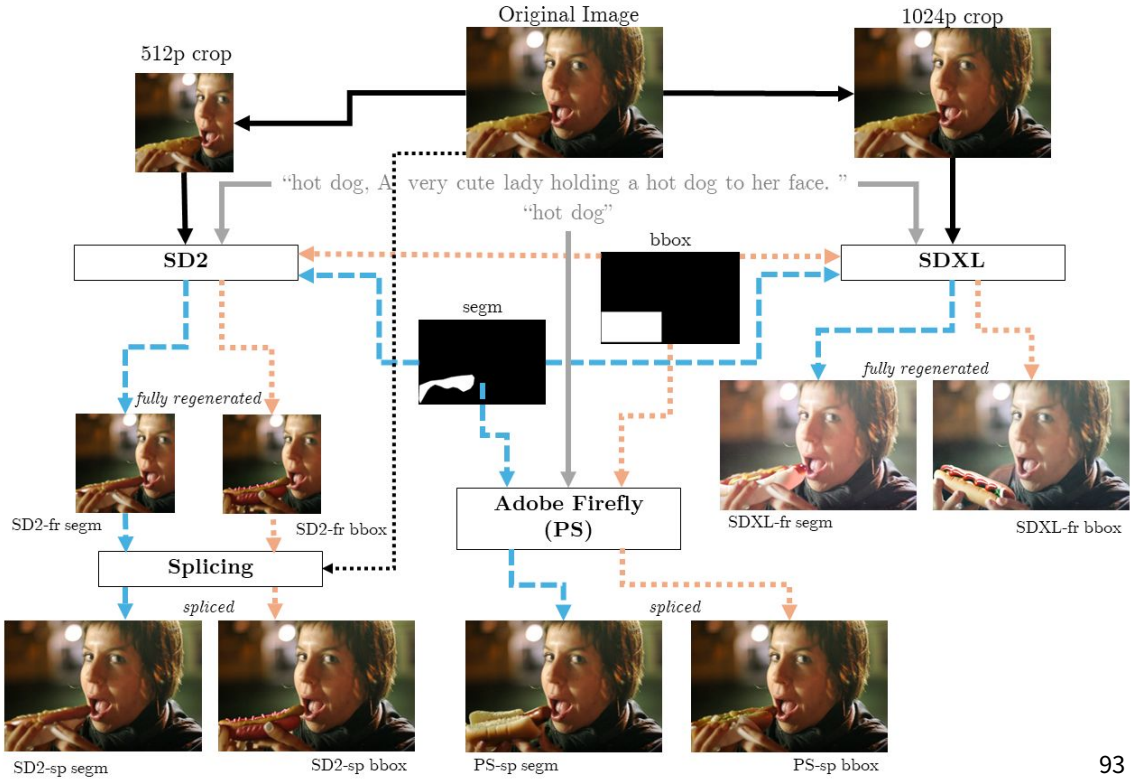
Dataset: TGIF

Text-Guided Inpainting Forgery

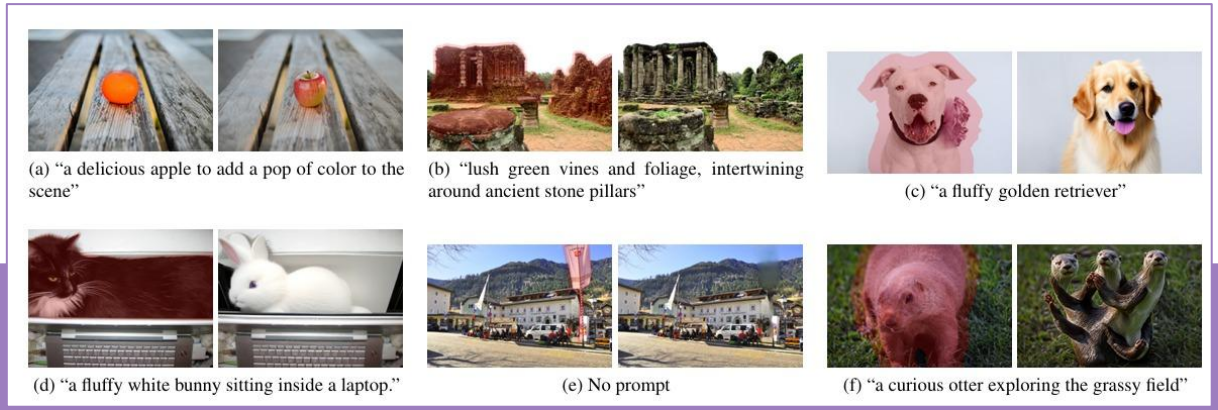
- around 80k images (in total)
  - ~3000 authentic, ~75000 manipulated
- manipulated using 3 different methods
- originate from MS-COCO
- supports localization
- TGIF2 extension

TGIF visually

- 3 methods
- crops
- segm vs bbox
- fully generated vs spliced



Dataset: SAGI-D


(a) “a delicious apple to add a pop of color to the scene”

(b) “lush green vines and foliage, intertwining around ancient stone pillars”

(c) “a fluffy golden retriever”

(d) “a fluffy white bunny sitting inside a laptop.”

(e) No prompt

(f) “a curious otter exploring the grassy field”

- Largest AI-inpainting dataset to date
  - ~95,800 inpainted images (MS-COCO, RAISE, OpenImages),
  - built over multiple SOTA inpainting models
- original image, inpainting mask, and prompt → supports localization
- tests for generalization

Dataset: SAGI-D

- Largest AI-inpainting dataset to date
  - ~95,800 inpainted images from ~78,700 sources (MS-COCO, RAISE, OpenImages), built over multiple SOTA inpainting models
- Ships each image with its original, inpainting mask, and prompt → supports localization, not just binary detection
- Built-in in-domain vs out-of-domain splits (different source images + different LLM) → directly tests generalization

PSCC-Net: Architecture

Top-down path:

- resolutions in parallel
- cross-scale information sharing

Bottom-up path:

- Localization
- Detection

SCCM → Spatio-Channel Correlation Module

- spatial correlation → compares regions
- channel-wise correlation (edges, colors, etc.)

separate detection head

Image inpainting



What is our research question?

Initial question
"What is the best small AI model for inpainting-based synthetic image detection?"

Scoping decisions
Localization, not just binary
Small = params, FLOPs, latency
Inpainting-specific, not generic AI

Refined research question
Which lightweight vision model best localizes inpainting-based image manipulations under compute constraints?

Research Methodology

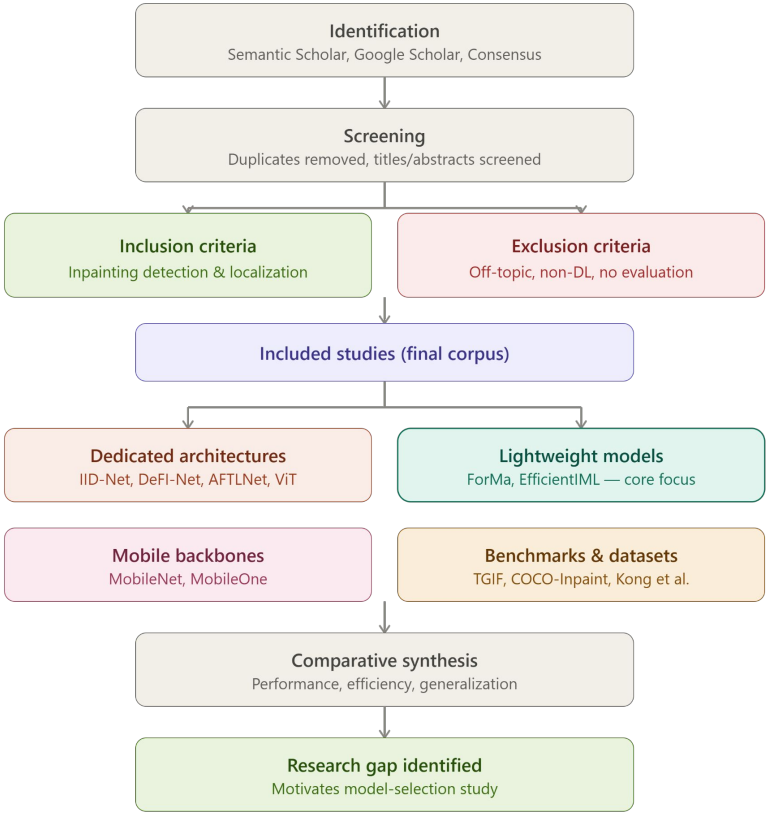


Figure: Systematic Literature Review pipeline from database search to the research gap motivating our model-selection study.

CNN-based models

Architecture:

- Identifies local spatial irregularities and boundary discontinuities
- Common architecture:
  - Enhancement
  - Feature Extraction (Encoder)
  - Decoder

Models:

- IID-Net (2022), IRL-Net (2023)

Pros:

- Good at detection
- Can have low complexity

Cons:

- Restricted receptive field
- Not great at generalization

Transformer-based models

Architecture:

- Patch Embedding — image divided into fixed-size patches, flattened and projected into visual tokens
- Self-Attention Encoder — every token directly compares itself to every other token in the image, capturing global relationships regardless of distance.
- Decoder — converts encoded tokens back into a pixel-level localization mask

Models:

- ML-ViT (2023)

Pros:

- Captures global context and structural consistency
- Attention maps offer some interpretability

Cons:

- High computational load
- Can miss fine-grained manipulation traces at boundaries

Frequency-domain models

Architecture:

- Transforms images into the frequency domain and look for irregularities

Models:

- Cat-Net

Pros:

- Sensitive to generative trace

Cons:

- Computation expensive,
- Not investigated on image inpainting

Diffusion-based models

Architecture:

- use the AI behind image generators (Diffusion model) as the detector
- feed the image back in, re-generates it and flags the parts that rebuild too perfectly
- common architecture:
    - Noising —> add noise to the image
    - Denoising —> rebuild it with a pretrained diffusion model
    - Compare —> reconstruction error / features → Decoder

Models:

- DIRE (2023), DiffForensics (2024), UGD-IML (2025)

Pros:

- uses the same technology that made the fake, so it's naturally tuned to AI edits

Cons:

- slow and heavy to run (a poor fit for small, fast models)

State Space Models (SSM)

Architecture:

- Replaces quadratic self-attention with continuous state space equations discretized into selective scanning mechanisms (e.g., Vision Mamba / VMamba 2D scanning)
- Captures long-range visual context and global pixel relationships across entire high-resolution images without memory bottlenecks or patch windowing constraints

Models:

- LoMa (Localization Mamba), IFDMamba (Context-Aware Mamba)

Pros:

- Linear computation complexity O(N), fast inference speed, and high-resolution scalability.

Cons:

- Emerging paradigm with fewer pre-trained forensic backbones compared to established CNNs/ViTs.

Next Steps: Building On Two Promising Architectures

ForMa

Guo et al., 2025

Lightweight image tampering localization built on Vision Mamba. Linear-complexity attention replaces quadratic self-attention.

SOTA generalization, lowest complexity, 10 datasets
general IML, not inpainting-specific

EfficientIML

Li et al., 2026

Designed explicitly for high-resolution diffusion inpainting localization, reducing the cost of large inputs.

Better IoU, FLOPs & speed than ViT baselines
most directly aligned with our task

Where current detectors break down

91% → 55%

Detectors rely on the global VAE/spectral shift from full regeneration, not the edited region; restoring the authentic background removes that cue and drops accuracy.

Source : Elif Nebioglu, Emirhan Bilgiç, and Adrian Popescu. "AI-Generated Image Detectors Overrely on Global Artifacts: Evidence from Inpainting Exchange."

>50% vs <20%

Detection holds for large inpainted regions but degrades sharply as the region shrinks, approaching chance for very small edits — accuracy scales with the manipulated-area ratio.

Source : Serafino Pandolfini, Lorenzo Pellegrini, Matteo Ferrara, and Davide Maltoni. "Detecting Localized Deepfakes: How Well Do Synthetic Image Detectors Handle Inpainting?"

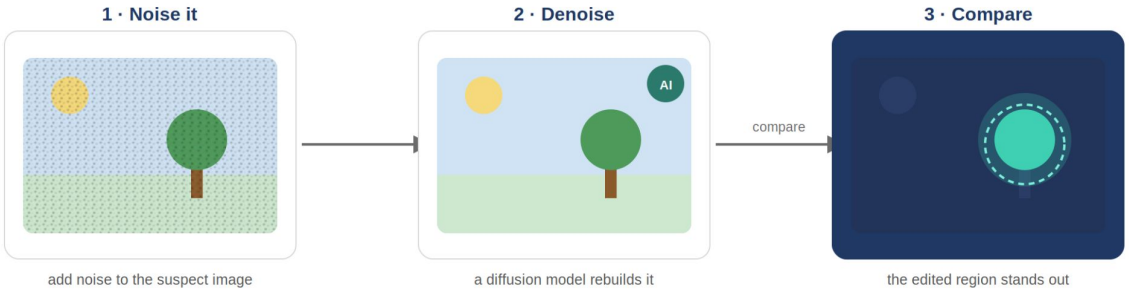
THE RESEARCH GAP

No study has benchmarked small models (ForMa, EfficientIML, PSCC-Net, mobile backbones) side-by-side for inpainting localization under matched parameters, latency, and metrics.

appendix

#	Method	Yr · Venue	Family	Task	Core idea / representation	Size (params / FLOPs)	Datasets	Key performance	Generalization	Fit for inpainting-localization	Paper link
1	Noiseprint	2020 · TIFS	Handcrafted (noise)	S / L	CNN camera-model noise fingerprint; manipulation = deviation from expected noise	DnCNN	camera / forensic sets	strong when noise stats intact	poor once image is re-decoded	Low alone; strong as a fused modality	https://arxiv.org/abs/1708.08396
2	Li, Luo & Huang	2017 · TIFS	Handcrafted (residual)	L	high-pass residual inconsistencies of classical (PDE) inpainting	—	classical inpainting	good on classical	fails on GAN/diffusion	Classical era only	https://doi.org/10.1109/TIFS.2017.2730822
3	Frequency / DCT / JPEG-grid (JPEG ghosts,	2009–21	Handcrafted (freq)	L	compression-grid & spectral inconsistencies	—	JPEG forensics	interpretable	brittle, generator-specific	Auxiliary only	— *(verify)*

appendix



The part the AI rebuilds perfectly is the part it made.

State Spatial Models (SSM)

- Replaces quadratic self-attention with continuous state space equations discretized into selective scanning mechanisms (e.g., Vision Mamba / VMamba 2D scanning)
- Captures long-range visual context and global pixel relationships across entire high-resolution images without memory bottlenecks or patch windowing constraints
- Linear computation complexity O(N), fast inference speed, and high-resolution scalability.

- Emerging paradigm with fewer pre-trained forensic backbones compared to established CNNs/ViTs.

Reminder of our goal

- We are not training new models and improving what’s esisintg
- we want a unified evaluation framework to truly assess which model is the best
- We diversify the search to different architectures while remaining under X M parameters